



INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE COMPUTO

Proyecto “Salud Mental”

Unidad de aprendizaje:

Desarrollo de aplicaciones móviles nativas

7CV2

Elaborado por:

Pardo Gomez Isaac

Jiménez Bernal Ernesto

Profesora:

Morales Guitron Sandra Luz

9 de enero de 2026

INSTITUTO POLITÉCNICO NACIONAL



ESCOM

9 de enero de 2026

1

ÍNDICE

INTRODUCCIÓN	4
OBJETIVO DEL PROYECTO	4
RESULTADOS ESPERADOS	4
TECNOLOGÍAS UTILIZADAS	5
<i>Lenguaje de Programación</i>	5
<i>Plataforma y SDK</i>	5
<i>Framework y Arquitectura</i>	5
<i>Gestión de Dependencias</i>	5
<i>Persistencia de Datos</i>	6
<i>Programación Asíncrona</i>	6
<i>Sistema de Notificaciones</i>	6
<i>Animaciones</i>	7
<i>Patrón Arquitectónico</i>	7
EXPLICACIÓN DETALLADA DE COMPONENTES IMPLEMENTADOS	7
<i>Sistema de Autenticación</i>	7
<i>Gestión de Sesión y Navegación</i>	7
<i>Interfaz de Usuario y Validación</i>	8
<i>Registro de Usuarios y Datos</i>	9
<i>Gestión de Datos con Room ORM</i>	10
<i>Gestión del Ciclo de Vida</i>	14
<i>Inicialización y Restauración</i>	14
<i>Preservación de Datos Volátiles</i>	15
<i>Control de Ejecución</i>	16
<i>Persistencia y Limpieza</i>	16
<i>Sistema de Persistencia con SharedPreferences</i>	17
<i>Operaciones de Lectura y Escritura</i>	18
<i>Gestión del Diario</i>	18
<i>Consideraciones y Escalabilidad</i>	18
<i>Sistema de Notificaciones</i>	19
<i>Gestión de Canales y Construcción</i>	19
<i>Programación de Alarmas</i>	20
<i>Recepción y Persistencia</i>	20
<i>Diario Personal</i>	21
<i>Creación Dinámica de Entradas</i>	21
<i>Visualización y Experiencia de Usuario (UX)</i>	22

<i>Funcionalidades Avanzadas</i>	22
<i>Gamificación</i>	23
<i>Ejercicios de Respiración</i>	24
<i>Arquitectura de la Animación y Guía Visual</i>	24
<i>Sincronización y Lógica de Control</i>	25
<i>Resiliencia ante el Ciclo de Vida (UX)</i>	26
<i>Persistencia y Gamificación</i>	26
<i>Meditación Guiada</i>	27
<i>Arquitectura de Dos Niveles</i>	27
<i>Experiencia de Sesión Inmersiva</i>	30
<i>Control y Gestión de Estados</i>	31
<i>Análisis y Gamificación</i>	32
<i>Mini-Juegos</i>	32
<i>Juego de Burbujas</i>	32
<i>Juego de Memoria</i>	34
<i>Juego de Colores</i>	35
<i>Sistema de Estadísticas y Logros</i>	37
<i>Manejo de Intents y Navegación</i>	38
RESULTADOS	39
CONCLUSIÓN	52

INTRODUCCIÓN

“Salud mental” es una plataforma móvil diseñada para el bienestar emocional que integra herramientas de autogestión y estimulación cognitiva bajo una arquitectura técnica sólida y moderna. El objetivo principal de la aplicación es proporcionar a los usuarios un entorno seguro y privado donde puedan documentar su estado emocional, practicar técnicas de respiración consciente y realizar ejercicios de distracción positiva a través de mini-juegos especializados. Este ecosistema se fundamenta en principios de la neurociencia y la psicología cognitiva, buscando reducir los niveles de cortisol y fortalecer la resiliencia mental mediante una interacción diaria y simplificada.

Desde la perspectiva tecnológica, la aplicación se ha construido siguiendo las mejores prácticas de desarrollo en Android, priorizando la eficiencia, la escalabilidad y una experiencia de usuario fluida. El sistema utiliza una estructura desacoplada que separa la lógica de negocio de la interfaz de usuario, garantizando que procesos complejos como la gestión de notificaciones, el seguimiento de estadísticas en tiempo real y la persistencia de datos operen sin comprometer el rendimiento del dispositivo. La integración de tecnologías como Room para el almacenamiento local y una gestión avanzada del ciclo de vida de las actividades asegura que cada sesión sea robusta, confiable y capaz de preservar el progreso del usuario ante cualquier circunstancia.

OBJETIVO DEL PROYECTO

“Salud mental” es una aplicación móvil nativa para Android diseñada para proporcionar herramientas digitales accesibles y efectivas para el manejo del estrés y el cuidado de la salud mental. El objetivo principal es ofrecer a los usuarios un espacio personal donde puedan practicar técnicas de relajación, llevar un registro emocional, y mantener hábitos saludables mediante recordatorios inteligentes y gamificación motivacional.

RESULTADOS ESPERADOS

- Facilitar la práctica regular de ejercicios de respiración controlada mediante guías visuales y temporizadores precisos.
- Promover la escritura reflexiva como herramienta terapéutica a través de un diario personal privado y estructurado.
- Ofrecer sesiones de meditación guiada con diferentes enfoques y duraciones adaptadas a distintas necesidades.
- Reducir el estrés mediante mini-juegos interactivos diseñados específicamente para distracción positiva.
- Mantener la adherencia del usuario mediante un sistema de notificaciones programadas no intrusivas.
- Fomentar la continuidad mediante gamificación con logros y seguimiento de

- progreso.
- Garantizar la privacidad total del usuario mediante almacenamiento local de datos sensibles.

TECNOLOGÍAS UTILIZADAS

Lenguaje de Programación

Kotlin 1.9.20 - Lenguaje oficial para desarrollo Android que proporciona null safety, funciones de extensión, coroutines, expresiones lambda, y una sintaxis más concisa y segura que Java. Se eligió por su interoperabilidad completa con el ecosistema Android y su capacidad de reducir significativamente el código boilerplate.

Plataforma y SDK

Android SDK API 24-34 - El proyecto soporta desde Android 7.0 Nougat (API 24) hasta Android 14 (API 34), cubriendo aproximadamente el 95% de los dispositivos Android activos. La API mínima 24 fue seleccionada para acceder a características modernas mientras se mantiene amplia compatibilidad.

Android Studio - Entorno de desarrollo integrado oficial para Android basado en IntelliJ IDEA, utilizado para la escritura de código, depuración, testing, y gestión del proyecto.

Framework y Arquitectura

- **AppCompat** - Biblioteca de compatibilidad hacia atrás para características modernas de Android.
- **Lifecycle Components** - Gestión consciente del ciclo de vida de Activities y Fragments.
- **ConstraintLayout** - Sistema de layout flexible y performante.

Material Design Components 1.11.0 - Implementación oficial de Material Design que proporciona componentes UI modernos como CardView, Snackbar, TextInputLayout, y gestión de temas consistente.

Gestión de Dependencias

Gradle 8.2 con Kotlin DSL - Sistema de build automatizado que gestiona dependencias, compilación, y empaquetado de la aplicación. El uso de Kotlin DSL en lugar de Groovy proporciona type-safety y mejor autocompletado en los scripts de build.

Persistencia de Datos

SharedPreferences - Sistema de almacenamiento clave-valor nativo de Android utilizado para datos de usuario, configuraciones, estadísticas, y contenido del diario. Se eligió por su simplicidad, velocidad, y porque los datos permanecen completamente locales garantizando privacidad absoluta.

Room Database 2.6.1 - Capa de abstracción sobre SQLite que proporciona verificación en tiempo de compilación de queries SQL, annotations para definición de esquemas, y soporte para coroutines. Implementado como alternativa escalable para gestión más robusta de datos relacionales.

Programación Asíncrona

Kotlin Coroutines 1.7.3 - Framework para programación asíncrona que permite operaciones no bloqueantes usando sintaxis secuencial. Utilizado para operaciones de I/O, acceso a base de datos, y tareas en segundo plano sin bloquear el hilo principal UI.

Lifecycle Runtime KTX 2.7.0 - Extensiones que integran coroutines con los lifecycle components, proporcionando scopes como lifecycleScope que automáticamente cancelan operaciones cuando el lifecycle termina.

Sistema de Notificaciones

AlarmManager - API del sistema Android para programar alarmas y eventos recurrentes incluso cuando la aplicación no está en ejecución. Utilizado para disparar notificaciones programadas en horarios específicos.

NotificationManager y NotificationCompat - APIs para crear, gestionar, y mostrar notificaciones. NotificationCompat proporciona compatibilidad hacia atrás para características de notificaciones modernas.

Notification Channels (API 26+) - Sistema que permite a los usuarios controlar granularmente qué tipos de notificaciones reciben y cómo se comportan, implementado mediante canales separados para recordatorios, motivación, y logros.

WorkManager 2.9.0 - Biblioteca Jetpack para tareas en segundo plano diferibles y garantizadas, utilizada opcionalmente para gestión más robusta de trabajo asíncrono.

Animaciones

Property Animation API - Sistema de animaciones basado en propiedades que permite animar cualquier propiedad de objetos. Utilizado mediante ValueAnimator y ObjectAnimator para animar escalas, rotaciones, alphas, y traducciones en ejercicios de respiración y meditación.

View Animation - Sistema de animaciones más simple utilizado para transiciones, fades, y efectos visuales en la interfaz de usuario.

Patrón Arquitectónico

Activity-Based Architecture - Arquitectura basada en Activities como componentes principales, apropiada para el alcance y complejidad del proyecto. Cada funcionalidad principal (login, diario, juegos, meditación) reside en su propia Activity con clara separación de responsabilidades.

Helper Classes Pattern - Clases auxiliares como NotificationHelper, NotificationScheduler, y MigrationHelper encapsulan lógica específica y reutilizable, promoviendo separación de concerns y testabilidad.

EXPLICACIÓN DETALLADA DE COMPONENTES IMPLEMENTADOS

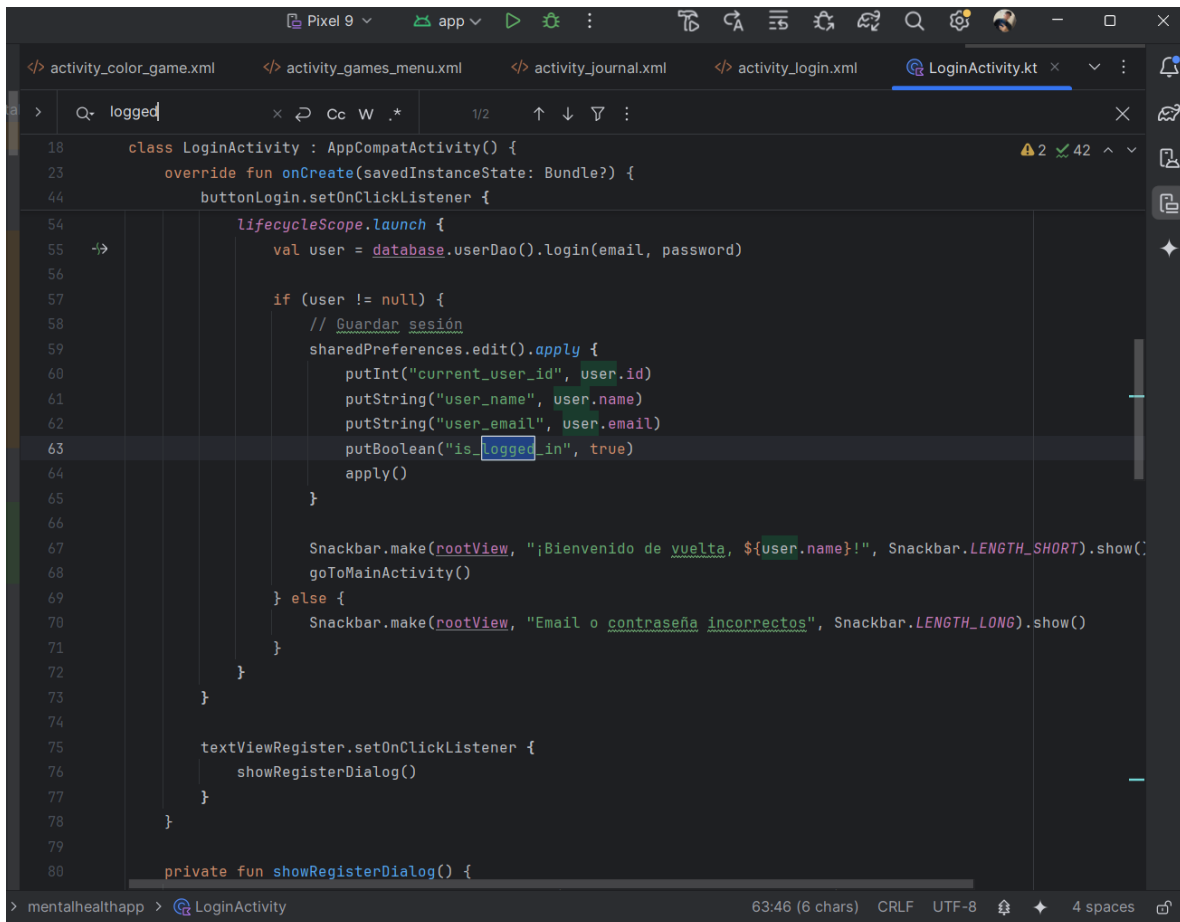
Sistema de Autenticación

El sistema gestiona el acceso mediante un flujo de registro y login basado en SharedPreferences, desacoplando la lógica de negocio de la interfaz de usuario.

Gestión de Sesión y Navegación

Punto de entrada: LoginActivity actúa como la actividad principal (LAUNCHER). Al iniciar, verifica el estado de la sesión mediante el flag is_logged_in.

Redirección automática: Si existe una sesión activa, el sistema navega hacia MainActivity y finaliza la actividad actual (finish()) para limpiar el back stack y optimizar la navegación.

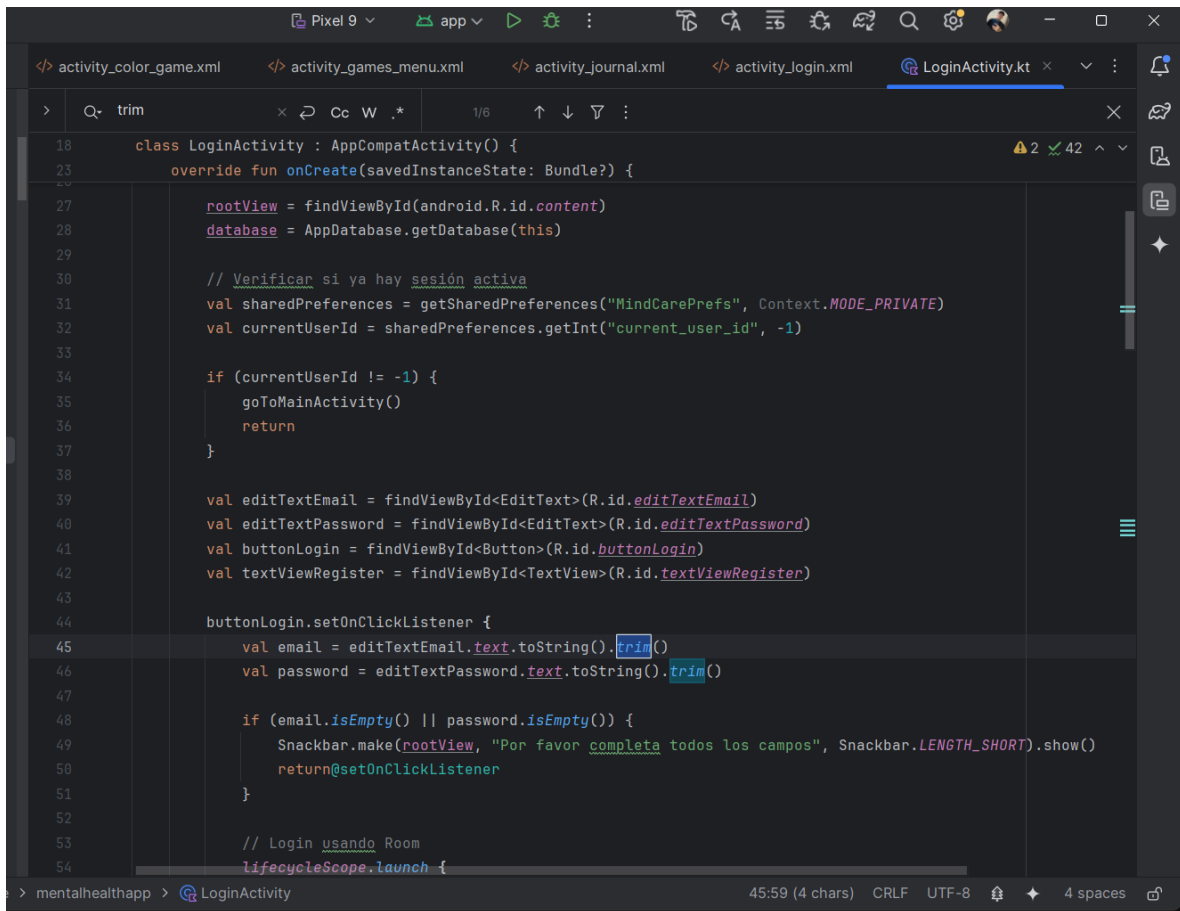


```
18 class LoginActivity : AppCompatActivity() {
23     override fun onCreate(savedInstanceState: Bundle?) {
44         buttonLogin.setOnClickListener {
54             lifecycleScope.launch {
55                 val user = database.userDao().login(email, password)
56
57                 if (user != null) {
58                     // Guardar sesión
59                     sharedPreferences.edit().apply {
60                         putInt("current_user_id", user.id)
61                         putString("user_name", user.name)
62                         putString("user_email", user.email)
63                         putBoolean("is_logged_in", true)
64                     }.apply()
65                 }
66
67                 Snackbar.make(rootView, "¡Bienvenido de vuelta, ${user.name}!", Snackbar.LENGTH_SHORT).show()
68                 goToMainActivity()
69             } else {
70                 Snackbar.make(rootView, "Email o contraseña incorrectos", Snackbar.LENGTH_LONG).show()
71             }
72         }
73     }
74
75     textViewRegister.setOnClickListener {
76         showRegisterDialog()
77     }
78 }
79
80 private fun showRegisterDialog() {
```

Interfaz de Usuario y Validación

Experiencia Visual: Se implementan TextInputLayout de Material Design para una estética moderna y Snackbars para proporcionar feedback interactivo en lugar de mensajes estáticos.

Seguridad de Inputs: La validación elimina espacios en blanco (trim()) y asegura que los campos cumplan con los criterios mínimos (longitud de caracteres y campos obligatorios).



```
18 class LoginActivity : AppCompatActivity() {
23     override fun onCreate(savedInstanceState: Bundle?) {
24         // Verificar si ya hay sesión activa
25         val sharedPreferences = getSharedPreferences("MindCarePrefs", Context.MODE_PRIVATE)
26         val currentUserId = sharedPreferences.getInt("current_user_id", -1)
27
28         if (currentUserId != -1) {
29             goToMainActivity()
30             return
31         }
32
33         val editTextEmail = findViewById<EditText>(R.id.editTextEmail)
34         val editTextPassword = findViewById<EditText>(R.id.editTextPassword)
35         val buttonLogin = findViewById<Button>(R.id.buttonLogin)
36         val textViewRegister = findViewById<TextView>(R.id.textViewRegister)
37
38         buttonLogin.setOnClickListener {
39             val email = editTextEmail.text.toString().trim()
40             val password = editTextPassword.text.toString().trim()
41
42             if (email.isEmpty() || password.isEmpty()) {
43                 Snackbar.make(rootView, "Por favor completa todos los campos", Snackbar.LENGTH_SHORT).show()
44                 return@setOnClickListener
45             }
46
47             // Login usando Room
48             lifecycleScope.launch {
```

Registro de Usuarios y Datos

Formulario: El registro se realiza mediante un AlertDialog personalizado que valida en cascada la integridad de los datos y la unicidad del correo electrónico.

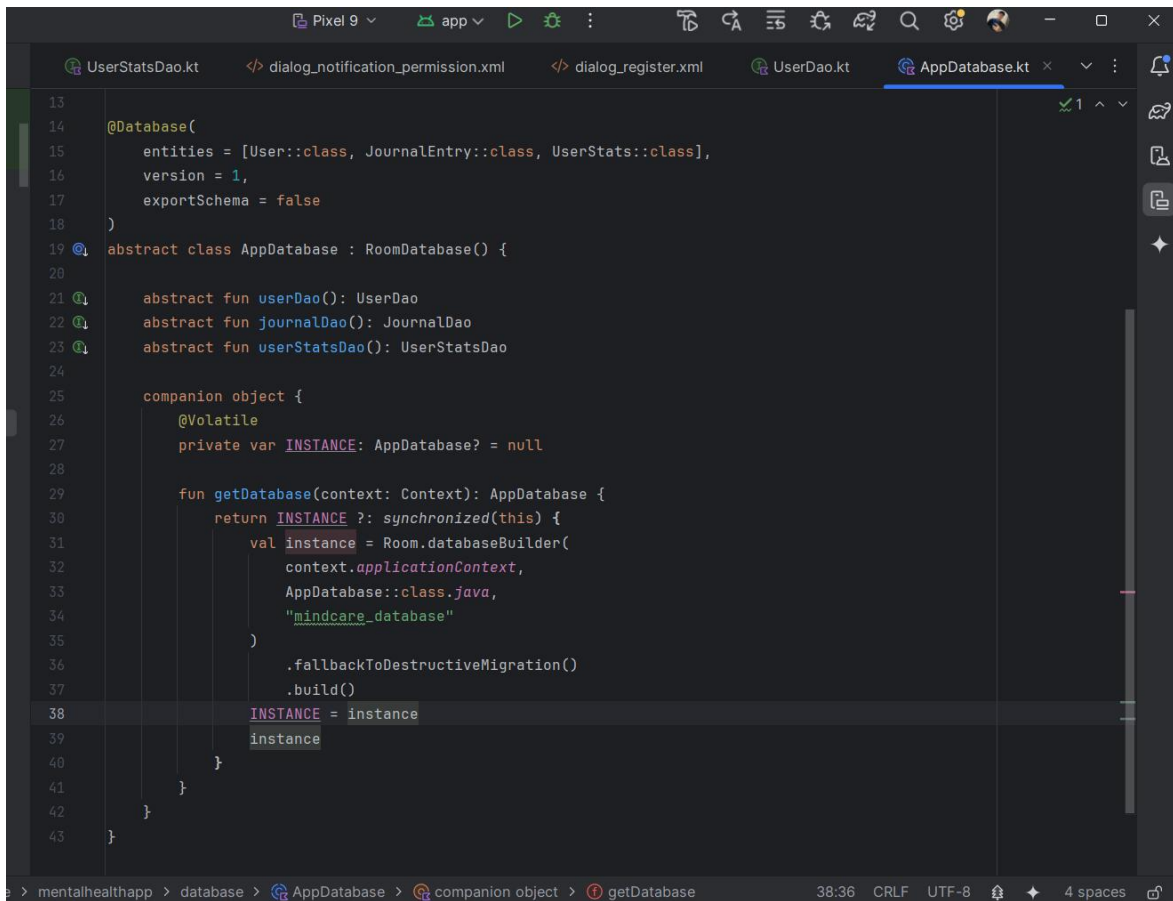
Persistencia: Al registrar un usuario, se genera un ID único basado en `currentTimeMillis()`. Se almacenan las credenciales y se inicializan contadores de actividad (diarios, respiración, etc.) con valores en cero.

```
18 class LoginActivity : AppCompatActivity() {
80     private fun showRegisterDialog() {
87         AlertDialog.Builder(this)
88             .setTitle("Crear cuenta")
89             .setView(dialogView)
90             .setPositiveButton("Registrar") { _, _ ->
91                 val name = editName.text.toString().trim()
92                 val email = editEmail.text.toString().trim()
93                 val password = editPassword.text.toString().trim()
94                 val confirmPassword = editConfirmPassword.text.toString().trim()
95
96                 when {
97                     name.isEmpty() || email.isEmpty() || password.isEmpty() -> {
98                         Snackbar.make(rootView, "Completa todos los campos", Snackbar.LENGTH_SHORT).show()
99                     }
100                     password != confirmPassword -> {
101                         Snackbar.make(rootView, "Las contraseñas no coinciden", Snackbar.LENGTH_SHORT).show()
102                     }
103                     password.length < 6 -> {
104                         Snackbar.make(rootView, "La contraseña debe tener al menos 6 caracteres", Snackbar.LENGTH_SHORT).show()
105                     }
106                     else -> {
107                         registerUser(name, email, password)
108                     }
109                 }
110             }
111             .setNegativeButton("Cancelar", null)
112             .show()
113     }
114 }
```

Este enfoque garantiza una experiencia de usuario fluida, eliminando la necesidad de re-autenticación constante y asegurando la persistencia de datos de forma ligera y eficiente.

Gestión de Datos con Room ORM

La infraestructura de datos del proyecto evoluciona hacia una solución profesional mediante la implementación de Room, una biblioteca de persistencia sobre SQLite que proporciona una capa de abstracción robusta y segura. El corazón de este sistema es la base de datos AppDatabase, configurada bajo el patrón Singleton para garantizar una única instancia compartida en toda la aplicación y evitar conflictos de memoria. Esta arquitectura utiliza entidades claramente definidas para representar a los usuarios, las entradas del diario y las estadísticas de progreso, integrando un mecanismo de migración destructiva que facilita la evolución del esquema durante el desarrollo.



```
13
14 @Database(
15     entities = [User::class, JournalEntry::class, UserStats::class],
16     version = 1,
17     exportSchema = false
18 )
19 abstract class AppDatabase : RoomDatabase() {
20
21     abstract fun userDao(): UserDao
22     abstract fun journalDao(): JournalDao
23     abstract fun userStatsDao(): UserStatsDao
24
25     companion object {
26         @Volatile
27         private var INSTANCE: AppDatabase? = null
28
29         fun getDatabase(context: Context): AppDatabase {
30             return INSTANCE ?: synchronized(this) {
31                 val instance = Room.databaseBuilder(
32                     context.applicationContext,
33                     AppDatabase::class.java,
34                     "mindcare_database"
35                 )
36                     .fallbackToDestructiveMigration()
37                     .build()
38                 INSTANCE = instance
39                 instance
40             }
41         }
42     }
43 }
```

La lógica de acceso a datos se centraliza en objetos de acceso a datos o DAO, que actúan como interfaces entre la base de datos y la lógica de negocio.

El UserDao gestiona la seguridad y el perfil, permitiendo operaciones críticas como la autenticación mediante consultas filtradas por credenciales y la unicidad de registros por correo electrónico.

```
1 package com.example.mentalhealthisapp.database.dao
2
3 import androidx.room.*
4 import com.example.mentalhealthisapp.database.entities.User
5
6 @Dao
7 interface UserDao {
8
9     @Insert(onConflict = OnConflictStrategy.REPLACE)
10    suspend fun insertUser(user: User): Long
11
12    @Update
13    suspend fun updateUser(user: User)
14
15    @Delete
16    suspend fun deleteUser(user: User)
17
18    @Query("SELECT * FROM users WHERE email = :email AND password = :password LIMIT 1")
19    suspend fun login(email: String, password: String): User?
20
21    @Query("SELECT * FROM users WHERE email = :email LIMIT 1")
22    suspend fun getUserByEmail(email: String): User?
23
24    @Query("SELECT * FROM users WHERE id = :userId LIMIT 1")
25    suspend fun getUserById(userId: Int): User?
26
27    @Query("UPDATE users SET lastMood = :mood WHERE id = :userId")
28    suspend fun updateMood(userId: Int, mood: String)
29
30    @Query("SELECT COUNT(*) FROM users")
31    suspend fun getUserCount(): Int
32 }
```

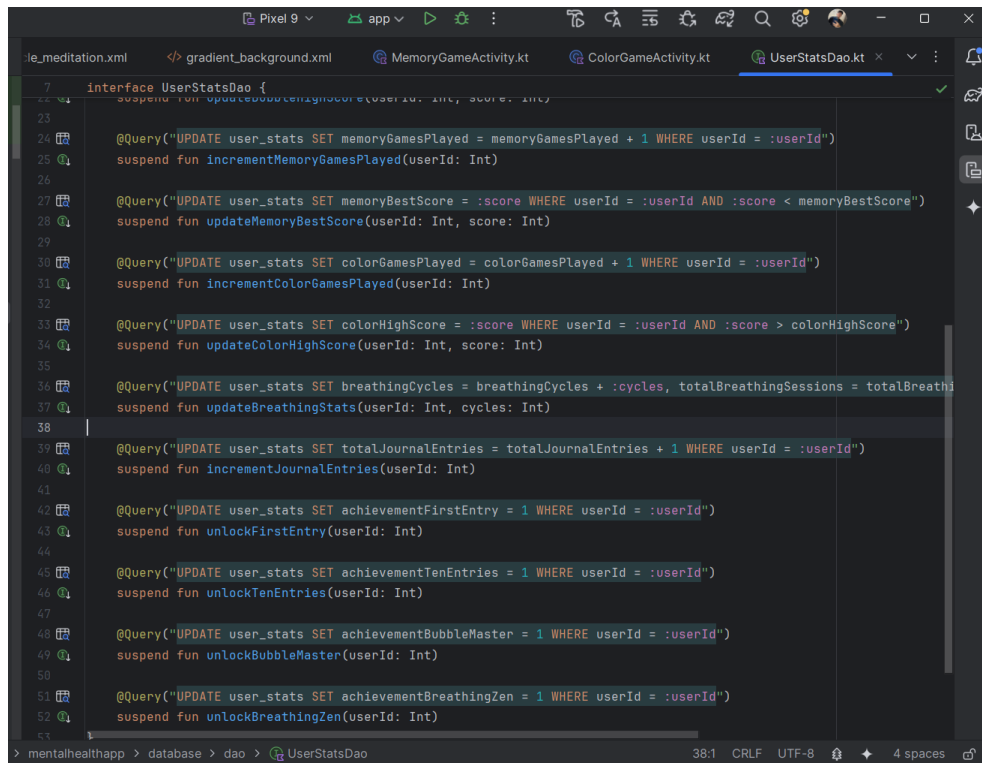
Por su parte, el JournalDao administra el historial de reflexiones del usuario, implementando un sistema de recuperación cronológica que ordena las entradas por marcas de tiempo. Este componente permite un control granular sobre el diario, desde la inserción de nuevas vivencias hasta la limpieza total de datos por usuario, asegurando que la información sea recuperada siempre de forma eficiente y ordenada.

```

1 package com.example.mentalhealthapp.database.dao
2
3 import androidx.room.*
4 import com.example.mentalhealthapp.database.entities.JournalEntry
5
6 @Dao
7 interface JournalDao {
8
9     @Insert
10    suspend fun insertEntry(entry: JournalEntry): Long
11
12    @Update
13    suspend fun updateEntry(entry: JournalEntry)
14
15    @Delete
16    suspend fun deleteEntry(entry: JournalEntry)
17
18    @Query("SELECT * FROM journal_entries WHERE userId = :userId ORDER BY timestamp DESC")
19    suspend fun getAllEntries(userId: Int): List<JournalEntry>
20
21    @Query("SELECT * FROM journal_entries WHERE userId = :userId AND id = :entryId LIMIT 1")
22    suspend fun getEntryById(userId: Int, entryId: Int): JournalEntry?
23
24    @Query("SELECT COUNT(*) FROM journal_entries WHERE userId = :userId")
25    suspend fun getEntryCount(userId: Int): Int
26
27    @Query("DELETE FROM journal_entries WHERE userId = :userId")
28    suspend fun deleteAllEntries(userId: Int)
29
30    @Query("SELECT * FROM journal_entries WHERE userId = :userId AND timestamp >= :startTime ORDER BY timestamp DESC")
31    suspend fun getEntriesSince(userId: Int, startTime: Long): List<JournalEntry>
32 }

```

El sistema de gamificación y seguimiento se potencia significativamente con el UserStatsDao, que introduce métodos atómicos para actualizar el progreso sin necesidad de reescribir objetos completos.



```
7 interface UserStatsDao {
8     @Query("UPDATE user_stats SET memoryGamesPlayed = memoryGamesPlayed + 1 WHERE userId = :userId")
9     suspend fun incrementMemoryGamesPlayed(userId: Int)
10
11     @Query("UPDATE user_stats SET memoryBestScore = :score WHERE userId = :userId AND :score < memoryBestScore")
12     suspend fun updateMemoryBestScore(userId: Int, score: Int)
13
14     @Query("UPDATE user_stats SET colorGamesPlayed = colorGamesPlayed + 1 WHERE userId = :userId")
15     suspend fun incrementColorGamesPlayed(userId: Int)
16
17     @Query("UPDATE user_stats SET colorHighScore = :score WHERE userId = :userId AND :score > colorHighScore")
18     suspend fun updateColorHighScore(userId: Int, score: Int)
19
20     @Query("UPDATE user_stats SET breathingCycles = breathingCycles + :cycles, totalBreathingSessions = totalBreathingSessions + :cycles")
21     suspend fun updateBreathingStats(userId: Int, cycles: Int)
22
23     @Query("UPDATE user_stats SET totalJournalEntries = totalJournalEntries + 1 WHERE userId = :userId")
24     suspend fun incrementJournalEntries(userId: Int)
25
26     @Query("UPDATE user_stats SET achievementFirstEntry = 1 WHERE userId = :userId")
27     suspend fun unlockFirstEntry(userId: Int)
28
29     @Query("UPDATE user_stats SET achievementTenEntries = 1 WHERE userId = :userId")
30     suspend fun unlockTenEntries(userId: Int)
31
32     @Query("UPDATE user_stats SET achievementBubbleMaster = 1 WHERE userId = :userId")
33     suspend fun unlockBubbleMaster(userId: Int)
34
35     @Query("UPDATE user_stats SET achievementBreathingZen = 1 WHERE userId = :userId")
36     suspend fun unlockBreathingZen(userId: Int)
37 }
```

A través de consultas SQL personalizadas, el sistema es capaz de incrementar contadores de juegos, actualizar récords personales comparando valores en tiempo real y desbloquear logros de manera permanente. Esta implementación garantiza la integridad de las estadísticas, desde los ciclos de respiración completados hasta las métricas de precisión en los mini-juegos, permitiendo que el progreso del usuario sea persistente, preciso y escalable ante el crecimiento de la aplicación.

Gestión del Ciclo de Vida

La robustez de la aplicación reside en la implementación estratégica de los callbacks del ciclo de vida, garantizando que la experiencia del usuario no se interrumpa ante cambios de estado o rotaciones de pantalla.

Inicialización y Restauración

Es el punto de partida donde se infla la interfaz y se configuran los listeners. Su función crítica es la restauración de estado: si existe un `savedInstanceState` (Bundle), la actividad recupera datos previos (puntuajes, fases de respiración o progreso de juegos) para continuar exactamente donde el usuario lo dejó.

```
</> activity_color_game.xml </> activity_games_menu.xml </> activity_journal.xml </> activity_login.xml LoginActivity.kt x v ⋮
> Q ONCREATE x ↺ Cc W .* 1/2 ↑ ↓ 🔍
18 class LoginActivity : AppCompatActivity() {
23 override fun onCreate(savedInstanceState: Bundle?) {
24     super.onCreate(savedInstanceState)
25     setContentView(R.layout.activity_login)
26
27     rootView = findViewById<android.R.id.content>()
28     database = AppDatabase.getDatabase(this)
29
30     // Verificar si ya hay sesión activa
31     val sharedPreferences = getSharedPreferences("MindCarePrefs", Context.MODE_PRIVATE)
32     val currentUserId = sharedPreferences.getInt("current_user_id", -1)
33
34     if (currentUserId != -1) {
35         goMainActivity()
36         return
37     }
38
39     val editTextEmail = findViewById<EditText>(R.id.editTextEmail)
40     val editTextPassword = findViewById<EditText>(R.id.editTextPassword)
41     val buttonLogin = findViewById<Button>(R.id.buttonLogin)
42     val textViewRegister = findViewById<TextView>(R.id.textViewRegister)
43
44     buttonLogin.setOnClickListener {
45         val email = editTextEmail.text.toString().trim()
46         val password = editTextPassword.text.toString().trim()
47
48         if (email.isEmpty() || password.isEmpty()) {
49             Snackbar.make(rootView, "Por favor completa todos los campos", Snackbar.LENGTH_SHORT).show()
50             return@setOnClickListener
51         }
52     }
53 }
```

Preservación de Datos Volátiles

Antes de una destrucción temporal (como al rotar el dispositivo), el sistema guarda información clave en un Bundle.

- **BreathingActivity**: Almacena la fase actual, ciclos completados y tiempo restante.
- **Juegos**: Guarda el puntaje, tiempo transcurrido y estado de la partida (ej. cartas volteadas).

```
Ⓢ override fun onSaveInstanceState(outState: Bundle) {
    super.onSaveInstanceState(outState)
    outState.putBoolean("isRunning", isRunning)
    outState.putInt("currentPhase", currentPhase)
    outState.putInt("cyclesCompleted", cyclesCompleted)
    outState.putLong("timeLeftInPhase", timeLeftInPhase)
}
```

Control de Ejecución

Estos métodos gestionan la atención del usuario:

- **onPause:** Detiene procesos inmediatos (timers y animaciones) para ahorrar recursos cuando la actividad pierde el foco.
- **onResume:** Evalúa el estado previo para reanudar automáticamente las tareas pausadas o esperar una nueva interacción.

```
override fun onPause() {  
    super.onPause()  
    // Pausar animaciones y timers si está corriendo  
    if (isRunning) {  
        isPaused = true  
        currentTimer?.cancel()  
        currentAnimator?.cancel()  
    }  
}  
  
override fun onResume() {  
    super.onResume()  
    // Reanudar ejercicio si estaba corriendo  
    if (isRunning && isPaused) {  
        isPaused = false  
        Snackbar.make(rootView, "Continuando ejercicio...", Snackbar.LENGTH_SHORT).show()  
        continueExercise()  
    }  
}
```

Persistencia y Limpieza

- **onStop:** Cuando la actividad deja de ser visible, se realizan operaciones de escritura pesadas en SharedPreferences, guardando estadísticas acumulativas (récores, burbujas reventadas, etc.). Esto separa la lógica de UI de la persistencia de datos para mejorar el rendimiento.
- **onDestroy:** Garantiza la liberación final de memoria, cancelando definitivamente timers y animadores para evitar fugas de memoria (memory leaks).

```

95  override fun onStop() {
96      super.onStop()
97      // Guardar progreso cuando la actividad no es visible
98      if (isRunning) {
99          val prefs = getSharedPreferences("MindCarePrefs", MODE_PRIVATE)
100         prefs.edit().apply {
101             putInt("breathing_cycles", cyclesCompleted)
102             putLong("breathing_last_session", System.currentTimeMillis())
103             apply()
104         }
105     }
106 }

107 override fun onDestroy() {
108     super.onDestroy()
109     stopExercise()
110 }

```

Sistema de Persistencia con SharedPreferences

SharedPreferences es un sistema de almacenamiento ligero tipo clave-valor que guarda datos en formato XML de forma privada (MODE_PRIVATE). Es ideal para preferencias de usuario, estadísticas rápidas y configuraciones.

Para mantener el orden, la aplicación organiza las claves jerárquicamente mediante prefijos:

```

18  class LoginActivity : AppCompatActivity() {
23      override fun onCreate(savedInstanceState: Bundle?) {
44          buttonLogin.setOnClickListener {
53              // Login usando Room
54              lifecycleScope.launch {
55                  val user = database.userDao().login(email, password)
56
57                  if (user != null) {
58                      // Guardar sesión
59                      sharedPreferences.edit().apply {
60                          putInt("current_user_id", user.id)
61                          putString("user_name", user.name)
62                          putString("user_email", user.email)
63                          putBoolean("is_logged_in", true)
64                          apply()
65                      }
66
67                      Snackbar.make(rootView, "¡Bienvenido de vuelta, ${user.name}!", Snackbar.LENGTH_SHORT).show()
68                      goToMainActivity()
69                  } else {
70                      Snackbar.make(rootView, "Email o contraseña incorrectos", Snackbar.LENGTH_LONG).show()
71                  }
72              }
73          }
74      }

```

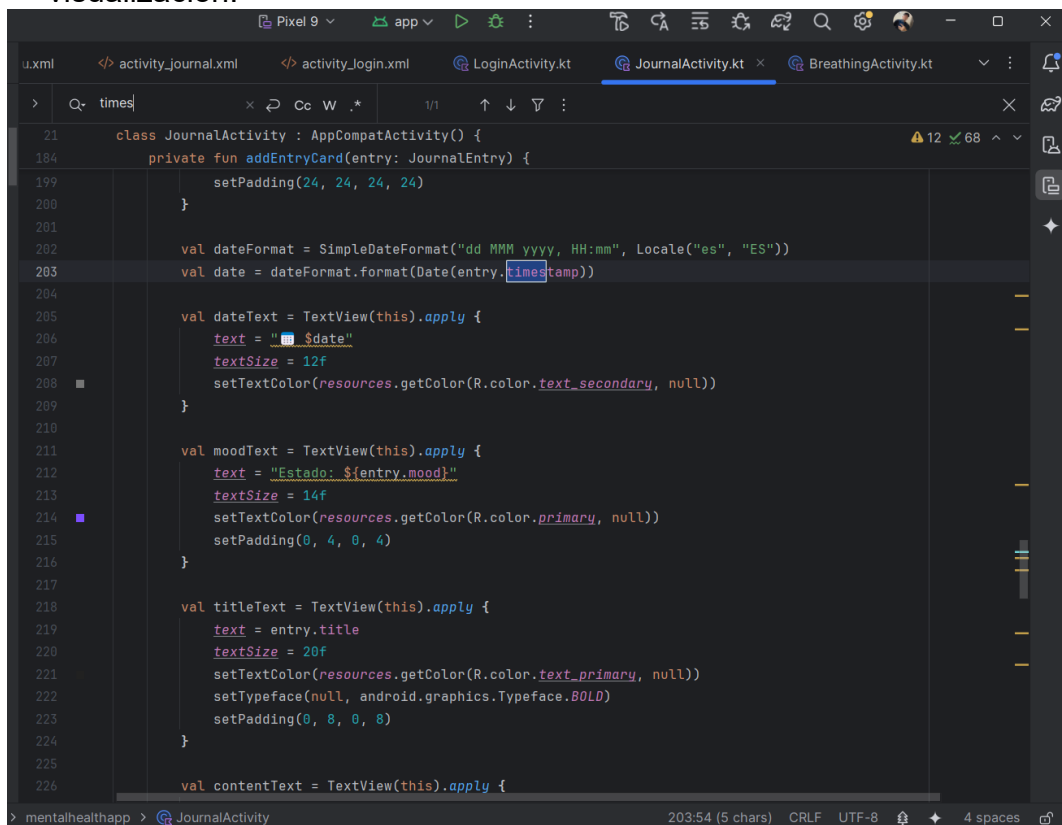
Operaciones de Lectura y Escritura

- **Escritura Asíncrona:** Se utiliza el patrón Editor. Se recomienda el método `apply()` sobre `commit()`, ya que `apply()` escribe en disco en segundo plano, evitando bloqueos en la interfaz de usuario (UI thread).
- **Lectura Segura:** Los métodos `get` (ej. `getInt()`) requieren un valor por defecto. Esto previene errores si la clave aún no existe, devolviendo valores lógicos como 0 o `false`.

Gestión del Diario

Para el diario se utiliza `StringSet`, almacenando entradas serializadas con un delimitador (`|`):

- **Formato:** "timestamp|título|contenido|humor".
- **Proceso:** Al recuperar los datos, el Set se convierte en una List, se divide cada cadena por el delimitador y se ordena cronológicamente para su visualización.



```
21 class JournalActivity : AppCompatActivity() {
184     private fun addEntryCard(entry: JournalEntry) {
199         setPadding(24, 24, 24, 24)
200     }
201
202     val dateFormat = SimpleDateFormat("dd MMM yyyy, HH:mm", Locale("es", "ES"))
203     val date = dateFormat.format(Date(entry.timestamp))
204
205     val dateText = TextView(this).apply {
206         text = "$date"
207         textSize = 12f
208         setTextColor(resources.getColor(R.color.text_secondary, null))
209     }
210
211     val moodText = TextView(this).apply {
212         text = "Estado: ${entry.mood}"
213         textSize = 14f
214         setTextColor(resources.getColor(R.color.text_primary, null))
215         setPadding(0, 4, 0, 4)
216     }
217
218     val titleText = TextView(this).apply {
219         text = entry.title
220         textSize = 20f
221         setTextColor(resources.getColor(R.color.text_primary, null))
222         setTypeface(null, android.graphics.Typeface.BOLD)
223         setPadding(0, 8, 0, 8)
224     }
225
226     val contentText = TextView(this).apply {
```

Consideraciones y Escalabilidad

Aunque eficiente, `SharedPreferences` presenta limitaciones importantes:

- **Modificabilidad:** Al usar `StringSet`, es necesario crear una instancia nueva del Set para asegurar que el sistema detecte cambios y guarde los datos correctamente.

- **Evolución:** No soporta consultas complejas (SQL) ni transacciones ACID. Para estructuras de datos más robustas o voluminosas, el proyecto está preparado para migrar a **Room Database**.

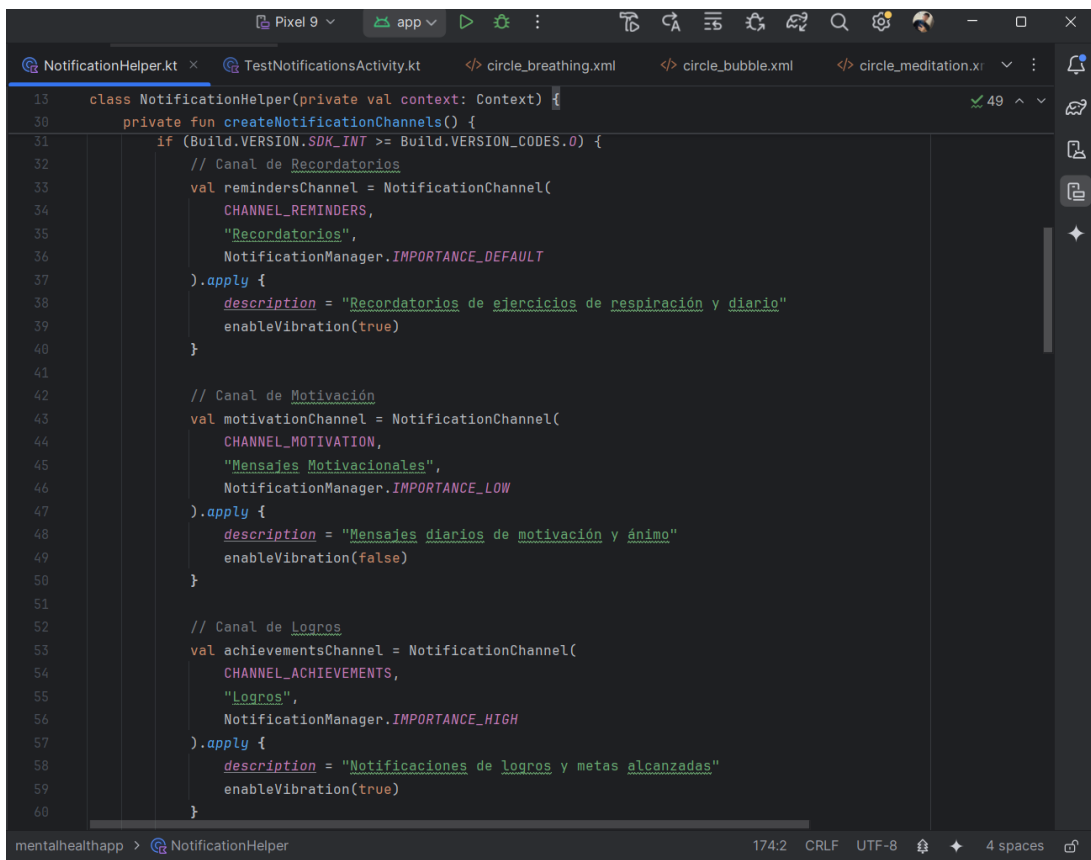
Sistema de Notificaciones

El sistema se basa en una estructura de tres componentes que garantiza la entrega de mensajes incluso si la aplicación no está en ejecución.

Gestión de Canales y Construcción

Desde Android 8.0, las notificaciones requieren Canales de Notificación para que el usuario pueda personalizar su prioridad y alertas.

- **Configuración:** Se definen canales específicos (Recordatorios, Motivación, Logros) con distintos niveles de importancia.
- **Construcción:** Se utiliza NotificationCompat.Builder para definir iconos, títulos, textos expandibles (BigTextStyle) y el comportamiento al tacto (setAutoCancel).
- **Interacción:** Mediante un PendingIntent (con flag FLAG_IMMUTABLE por seguridad), la notificación permite al usuario navegar directamente a una actividad específica.



```

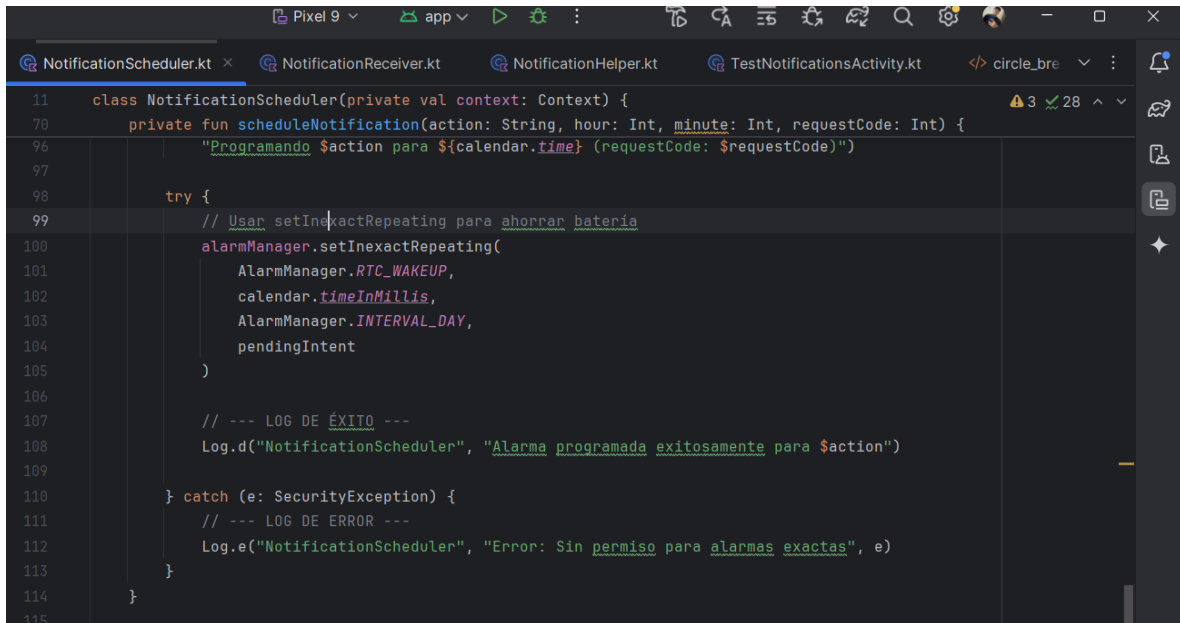
13 class NotificationHelper(private val context: Context) {
14     private fun createNotificationChannels() {
15         if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
16             // Canal de Recordatorios
17             val remindersChannel = NotificationChannel(
18                 CHANNEL_REMINDERS,
19                 "Recordatorios",
20                 NotificationManager.IMPORTANCE_DEFAULT
21             ).apply {
22                 description = "Recordatorios de ejercicios de respiración y diario"
23                 enableVibration(true)
24             }
25
26             // Canal de Motivación
27             val motivationChannel = NotificationChannel(
28                 CHANNEL_MOTIVATION,
29                 "Mensajes Motivacionales",
30                 NotificationManager.IMPORTANCE_LOW
31             ).apply {
32                 description = "Mensajes diarios de motivación y ánimo"
33                 enableVibration(false)
34             }
35
36             // Canal de Logros
37             val achievementsChannel = NotificationChannel(
38                 CHANNEL_ACHIEVEMENTS,
39                 "Logros",
40                 NotificationManager.IMPORTANCE_HIGH
41             ).apply {
42                 description = "Notificaciones de logros y metas alcanzadas"
43                 enableVibration(true)
44             }
45         }
46     }
47 }

```

Programación de Alarmas

Para ejecutar tareas en momentos exactos, se utiliza el AlarmManager:

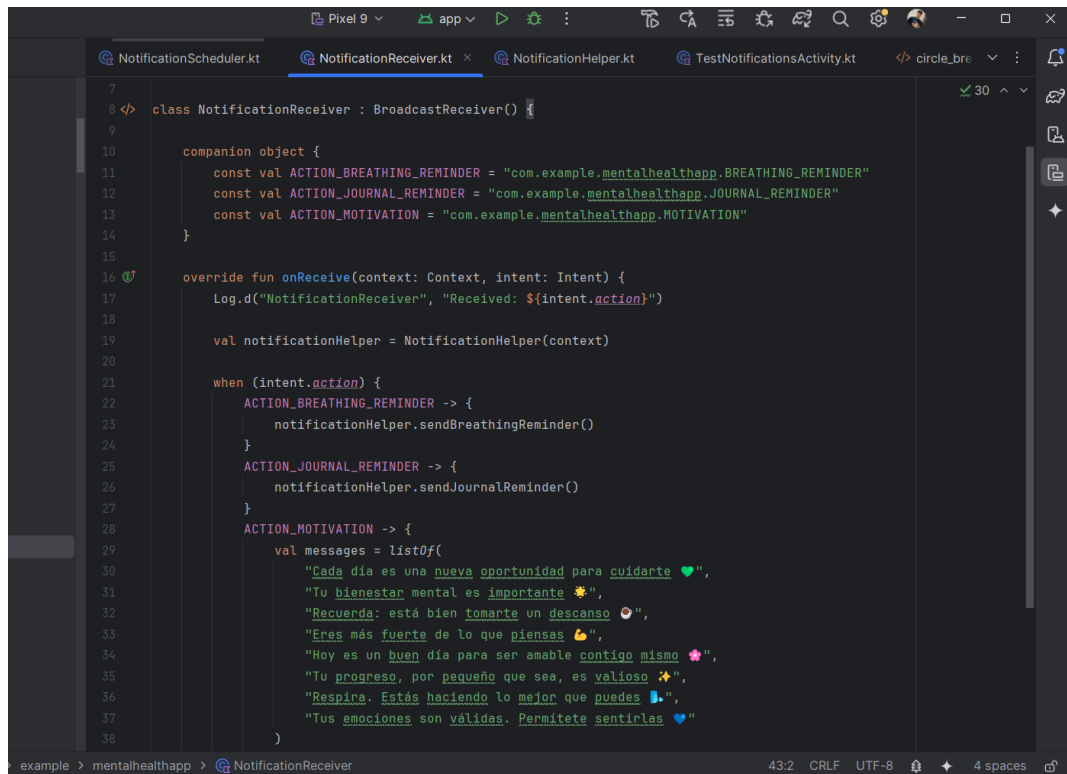
- **Eficiencia Energética:** Se emplea `setInexactRepeating` para agrupar alarmas de diferentes apps y reducir el consumo de batería.
- **Lógica Temporal:** Utiliza la clase `Calendar` para programar alertas diarias, recalculando automáticamente el disparo para el día siguiente si la hora programada ya pasó.



```
11 class NotificationScheduler(private val context: Context) {
70     private fun scheduleNotification(action: String, hour: Int, minute: Int, requestCode: Int) {
96         "Programando $action para ${calendar.time} (requestCode: $requestCode)"
97
98         try {
99             // Usar setInexactRepeating para ahorrar batería
100             alarmManager.setInexactRepeating(
101                 AlarmManager.RTC_WAKEUP,
102                 calendar.timeInMillis,
103                 AlarmManager.INTERVAL_DAY,
104                 pendingIntent
105             )
106
107             // --- LOG DE ÉXITO ---
108             Log.d("NotificationScheduler", "Alarma programada exitosamente para $action")
109
110         } catch (e: SecurityException) {
111             // --- LOG DE ERROR ---
112             Log.e("NotificationScheduler", "Error: Sin permiso para alarmas exactas", e)
113         }
114     }
115 }
```

Recepción y Persistencia

- **NotificationReceiver:** Actúa como un puente que intercepta la alarma, decide qué tipo de mensaje mostrar (seleccionando frases aleatorias para mayor dinamismo) y solicita al NotificationHelper que dispare la notificación.
- **Resiliencia (BootReceiver):** Dado que las alarmas se borran al apagar el dispositivo, este componente escucha el evento `ACTION_BOOT_COMPLETED` para reprogramar automáticamente todas las alertas configuradas previamente por el usuario.



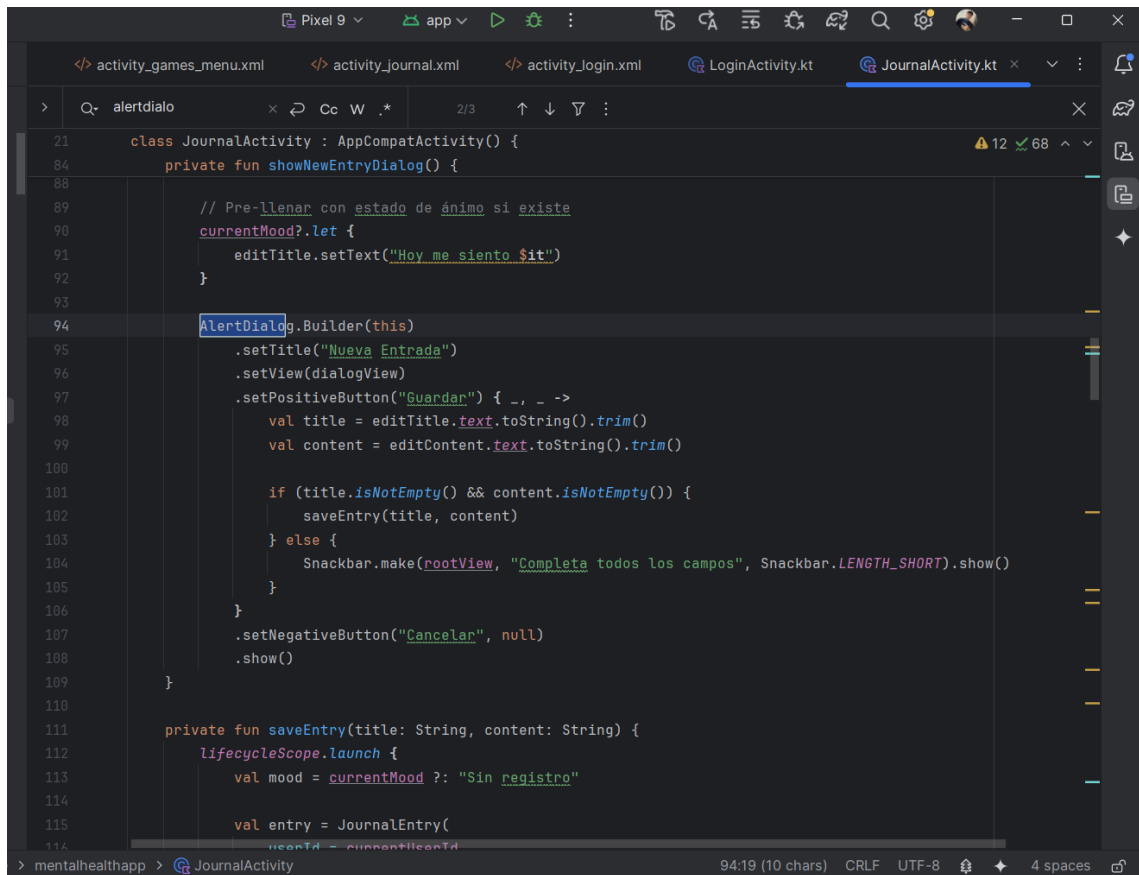
```
7
8 <> class NotificationReceiver : BroadcastReceiver() {
9
10     companion object {
11         const val ACTION_BREATHING_REMINDER = "com.example.mentalhealtheapp.BREATHING_REMINDER"
12         const val ACTION_JOURNAL_REMINDER = "com.example.mentalhealtheapp.JOURNAL_REMINDER"
13         const val ACTION_MOTIVATION = "com.example.mentalhealtheapp.MOTIVATION"
14     }
15
16     override fun onReceive(context: Context, intent: Intent) {
17         Log.d("NotificationReceiver", "Received: ${intent.action}")
18
19         val notificationHelper = NotificationHelper(context)
20
21         when (intent.action) {
22             ACTION_BREATHING_REMINDER -> {
23                 notificationHelper.sendBreathingReminder()
24             }
25             ACTION_JOURNAL_REMINDER -> {
26                 notificationHelper.sendJournalReminder()
27             }
28             ACTION_MOTIVATION -> {
29                 val messages = listOf(
30                     "Cada día es una nueva oportunidad para cuidarte 💚",
31                     "Tu bienestar mental es importante 🌟",
32                     "Recuerda: está bien tomarte un descanso 🛌",
33                     "Eres más fuerte de lo que piensas 💪",
34                     "Hoy es un buen día para ser amable contigo mismo 🌸",
35                     "Tu progreso, por pequeño que sea, es valioso ✨",
36                     "Respira. Estás haciendo lo mejor que puedes 🌊",
37                     "Tus emociones son válidas. Permítele sentir las 💙"
38                 )
39             }
40         }
41     }
42 }
```

Diario Personal

El diario permite una gestión completa de entradas (CRUD) vinculando texto libre con estados emocionales y marcas de tiempo, todo persistido de manera ligera.

Creación Dinámica de Entradas

- **Interfaz Inteligente:** Mediante un AlertDialog personalizado, el sistema facilita la escritura con áreas de texto multilínea. Si el usuario seleccionó un estado de ánimo previamente, el título se genera automáticamente (ej. "Hoy me siento Feliz").
- **Serialización:** Los datos se empaquetan en una cadena delimitada (timestamp|título|contenido|humor) y se guardan asincrónicamente en un StringSet para no afectar el rendimiento de la aplicación.



```
21 class JournalActivity : AppCompatActivity() {
84     private fun showNewEntryDialog() {
88         // Pre-llenar con estado de ánimo si existe
89         currentMood?.let {
90             editTitle.setText("Hoy me siento $it")
91         }
92     }
93
94     AlertDialog.Builder(this)
95         .setTitle("Nueva Entrada")
96         .setView(dialogView)
97         .setPositiveButton("Guardar") { _, _ ->
98             val title = editTitle.text.toString().trim()
99             val content = editContent.text.toString().trim()
100
101             if (title.isNotEmpty() && content.isNotEmpty()) {
102                 saveEntry(title, content)
103             } else {
104                 Snackbar.make(rootView, "Completa todos los campos", Snackbar.LENGTH_SHORT).show()
105             }
106         }
107         .setNegativeButton("Cancelar", null)
108         .show()
109 }
110
111 private fun saveEntry(title: String, content: String) {
112     lifecycleScope.launch {
113         val mood = currentMood ?: "Sin registro"
114
115         val entry = JournalEntry(
116             moodId = currentMoodId,
```

Visualización y Experiencia de Usuario (UX)

Al no conocerse el número de entradas, la app construye CardViews programáticamente. Cada tarjeta incluye una previsualización del contenido (máximo 3 líneas) y metadatos como fecha e iconos representativos.

El sistema convierte el conjunto de datos en una lista para ordenarla, asegurando que las reflexiones más recientes aparezcan primero.

Funcionalidades Avanzadas

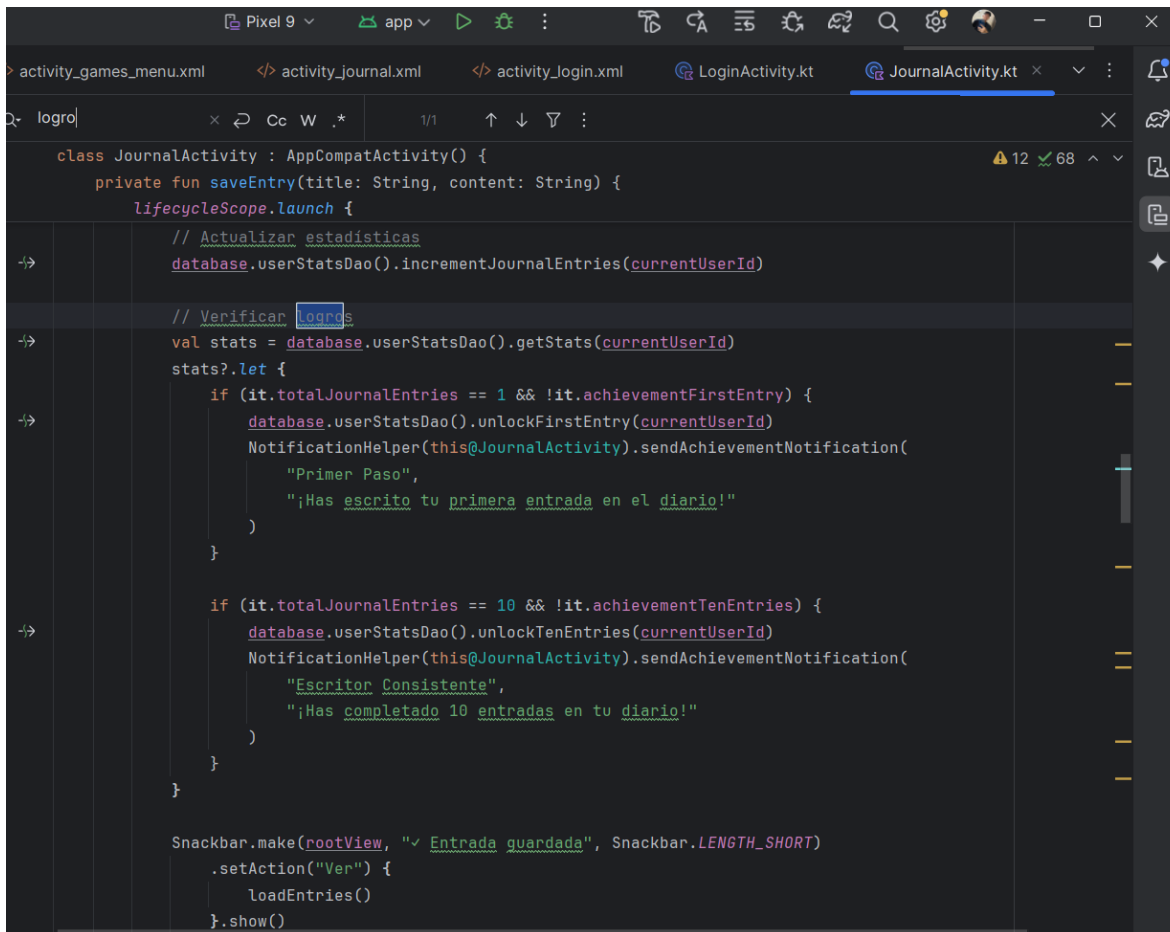
Utiliza el sistema nativo de Android (ACTION_SEND) para exportar entradas a cualquier aplicación externa (WhatsApp, Keep, Email) con un formato profesional que incluye pie de página personalizado.

Para evitar pérdidas accidentales, al borrar una entrada se activa un **Snackbar** con la opción de restaurarla. Este patrón de diseño mejora la confianza del usuario en la interfaz.

```
Pixel 9  app  1/1  JournalActivity.kt
>  Q- action_send  x ↺ Cc W .*  1/1  ↑ ↓ 🔍
21  class JournalActivity : AppCompatActivity() {
287  private fun shareEntry(entry: JournalEntry, date: String) {
288      val shareIntent = Intent(Intent.ACTION_SEND).apply {
289          type = "text/plain"
290          putExtra(Intent.EXTRA_SUBJECT, "Mi Diario - ${entry.title}")
291          putExtra(Intent.EXTRA_TEXT, "📅 ${entry.title}\n📅 $date\n\n${entry.content}\n\n✨ Escrito en MindCare")
292      }
293      startActivity(Intent.createChooser(shareIntent, "Compartir entrada"))
294  }
295
296  private fun deleteEntry(entry: JournalEntry) {
297      AlertDialog.Builder(this)
298          .setTitle("Eliminar entrada")
299          .setMessage("¿Estás seguro de eliminar esta entrada?")
300          .setPositiveButton("Eliminar") { _, _ ->
301              lifecycleScope.launch {
302                  database.journalDao().deleteEntry(entry)
303
304                  Snackbar.make(rootView, "Entrada eliminada", Snackbar.LENGTH_LONG)
305                      .setAction("Deshacer") {
306                          lifecycleScope.launch {
307                              database.journalDao().insertEntry(entry)
308                              loadEntries()
309                          }
310                      }.show()
311
312                  loadEntries()
313              }
314          }
315          .setNegativeButton("Cancelar", null)
316      }
317  }
```

Gamificación

El diario funciona bajo una arquitectura orientada a eventos. Al guardar una entrada, se dispara una verificación inmediata del contador global. Si se cumplen hitos (como la primera entrada o llegar a diez), el sistema desbloquea automáticamente el logro correspondiente y emite una notificación de "Logro Desbloqueado" sin necesidad de procesos en segundo plano constantes.



```
class JournalActivity : AppCompatActivity() {
    private fun saveEntry(title: String, content: String) {
        lifecycleScope.launch {
            // Actualizar estadísticas
            database.userStatsDao().incrementJournalEntries(currentUserId)

            // Verificar logs
            val stats = database.userStatsDao().getStats(currentUserId)
            stats?.let {
                if (it.totalJournalEntries == 1 && !it.achievementFirstEntry) {
                    database.userStatsDao().unlockFirstEntry(currentUserId)
                    NotificationHelper(this@JournalActivity).sendAchievementNotification(
                        "Primer Paso",
                        "¡Has escrito tu primera entrada en el diario!"
                    )
                }

                if (it.totalJournalEntries == 10 && !it.achievementTenEntries) {
                    database.userStatsDao().unlockTenEntries(currentUserId)
                    NotificationHelper(this@JournalActivity).sendAchievementNotification(
                        "Escritor Consistente",
                        "¡Has completado 10 entradas en tu diario!"
                    )
                }
            }
        }

        Snackbar.make(rootView, "✓ Entrada guardada", Snackbar.LENGTH_SHORT)
            .setAction("Ver") {
                loadEntries()
            }.show()
    }
}
```

Ejercicios de Respiración

Este módulo implementa la técnica Box Breathing, diseñada para reducir el estrés mediante la activación del sistema nervioso parasimpático a través de un ciclo rítmico de 12 segundos.

Arquitectura de la Animación y Guía Visual

La interfaz utiliza una representación minimalista para guiar al usuario sin distracciones:

- **Dinámica de Escala:** Mediante ValueAnimator, un círculo central se expande (escala 1.0 a 2.0) durante la **inhala**ción y se contrae durante la **exhalación**. En la fase de **retención**, el círculo permanece estático, proporcionando una referencia visual clara del estado pulmonar esperado.
- **Código de Colores e Instrucciones:** Para reforzar el *feedback*, las instrucciones cambian de color según la fase: **Verde** (Inhalar), **Morado** (Retener) y **Azul** (Exhalar).

```

16 class BreathingActivity : AppCompatActivity() {
174     private fun breatheInCycle() {
188     }
189
190     private fun breatheIn(onComplete: () -> Unit) {
191         textInstruction.text = "🌊 Inhala profundamente"
192         textInstruction.setTextColor(ContextCompat.getColor(this, R.color.accent))
193
194         animateCircle(1f, 2f, 4000)
195         countdown(4, onComplete)
196     }
197
198     private fun hold(onComplete: () -> Unit) {
199         textInstruction.text = "🌊 Sostén la respiración"
200         textInstruction.setTextColor(ContextCompat.getColor(this, R.color.primary))
201
202         countdown(4, onComplete)
203     }
204
205     private fun breatheOut(onComplete: () -> Unit) {
206         textInstruction.text = "🌊 Exhala lentamente"
207         textInstruction.setTextColor(ContextCompat.getColor(this, android.R.color.holo_blue_dark))
208
209         animateCircle(2f, 1f, 4000)
210         countdown(4, onComplete)
211     }
212
213     private fun animateCircle(fromScale: Float, toScale: Float, duration: Long) {

```

Sincronización y Lógica de Control

Precisión Temporal: Se emplea un `CountDownTimer` con intervalos de un segundo para sincronizar el contador numérico, el texto instructivo y la animación del círculo.

Recursividad Controlada: El ciclo completo se gestiona mediante llamadas encadenadas (Inhale → Hold → Exhale), permitiendo una transición fluida y continua entre las repeticiones.

```

private fun countdown(seconds: Int, onComplete: () -> Unit) {
    currentTimer?.cancel()
    currentTimer = object : CountDownTimer((seconds * 1000).toLong(), 1000) {
        override fun onTick(millisUntilFinished: Long) {
            timeLeftInPhase = millisUntilFinished
            val secondsLeft = (millisUntilFinished / 1000).toInt() + 1
            textCounter.text = secondsLeft.toString()
        }

        override fun onFinish() {
            if (!isPaused) {
                onComplete()
            }
        }
    }.start()
}

```

Resiliencia ante el Ciclo de Vida (UX)

El ejercicio está diseñado para ser robusto frente a interrupciones o cambios de orientación:

- **Pausa Inteligente:** En onPause, se detienen los temporizadores y animadores para ahorrar recursos.
- **Restauración de Estado:** Gracias a onSaveInstanceState, la actividad recuerda exactamente en qué fase y segundo se encontraba el usuario, permitiendo reanudar la sesión sin perder el progreso tras una rotación de pantalla o cambio de aplicación.

```
4
5  override fun onPause() {
6      super.onPause()
7      // Pausar animaciones y timers si está corriendo
8      if (isRunning) {
9          isPaused = true
10         currentTimer?.cancel()
11         currentAnimator?.cancel()
12     }
13 }
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57  override fun onSaveInstanceState(outState: Bundle) {
58      super.onSaveInstanceState(outState)
59      outState.putBoolean("isRunning", isRunning)
60      outState.putInt("currentPhase", currentPhase)
61      outState.putInt("cyclesCompleted", cyclesCompleted)
62      outState.putLong("timeLeftInPhase", timeLeftInPhase)
63  }
```

Persistencia y Gamificación

Al finalizar o detener la sesión, el sistema realiza las siguientes acciones en segundo plano:

- **Registro de Datos:** Actualiza en SharedPreferences el conteo total de ciclos y sesiones completadas.
- **Disparo de Eventos:** Verifica hitos (como alcanzar las 5 sesiones) para desbloquear logros y muestra un resumen estadístico al usuario mediante un Snackbar, reforzando el hábito de bienestar.

```
Pixel 9  app  BreathingActivity.kt
class BreathingActivity : AppCompatActivity() {
    private fun stopExercise() {
        if (cyclesCompleted > 0) {
            // Guardar estadísticas en la base de datos
            val sharedPreferences = getSharedPreferences("MindCarePrefs", MODE_PRIVATE)
            val currentUserId = sharedPreferences.getInt("current_user_id", -1)

            if (currentUserId != -1) {
                lifecycleScope.launch {
                    val database = AppDatabase.getDatabase(this@BreathingActivity)
                    database.userStatsDao().updateBreathingStats(currentUserId, cyclesCompleted)

                    // Verificar logro
                    val stats = database.userStatsDao().getStats(currentUserId)
                    stats?.let {
                        if (it.totalBreathingSessions >= 5 && !it.achievementBreathingZen) {
                            database.userStatsDao().unlockBreathingZen(currentUserId)
                            NotificationHelper(this@BreathingActivity).sendAchievementNotification(
                                "Respirador Zen",
                                "¡Has completado 5 sesiones de respiración!"
                            )
                        }
                    }
                }
            }

            Snackbar.make(
                rootView,
                "Completaste $cyclesCompleted ciclos. ¡Bien hecho! 🌟",
                Snackbar.LENGTH_LONG
            ).setAction("Compartir") {

```

Meditación Guiada

El sistema se basa en un catálogo diversificado y sesiones inmersivas que combinan guía visual, auditiva y cronometrada para fomentar el bienestar.

Arquitectura de Dos Niveles

La aplicación separa las responsabilidades para optimizar el rendimiento:

- **MeditationActivity (Catálogo):** Gestiona la oferta de 6 tipos de sesiones (Principiantes, Ansiedad, Enfoque, etc.). Utiliza creación dinámica de tarjetas bajo el principio **DRY** (*Don't Repeat Yourself*), lo que permite generar múltiples opciones con un solo diseño base, variando colores, duraciones y metadatos.

```
Pixel 9  app  MeditationActivity.kt
class MeditationActivity : AppCompatActivity() {
    private fun setupMeditationCards() {
        // Meditación para principiantes
        addMeditationCard(
            "🧘 Meditación para Principiantes",
            "Perfecta si es tu primera vez meditando",
            "5 minutos",
            "#E1BEE7",
            5
        )

        // Meditación para dormir
        addMeditationCard(
            "🌙 Meditación para Dormir",
            "Relájate profundamente antes de dormir",
            "10 minutos",
            "#B2EBF2",
            10
        )

        // Meditación anti-ansiedad
        addMeditationCard(
            "🧘 Calma la Ansiedad",
            "Reduce la ansiedad y encuentra paz interior",
            "7 minutos",
            "#CSE1A5",
            7
        )

        // Meditación de gratitud
    }
}
```

- **MeditationSessionActivity (Sesión):** Una interfaz dedicada y aislada que permite animaciones fluidas y una experiencia inmersiva sin interferencias de navegación.

```

class MeditationSessionActivity : AppCompatActivity() {

    private fun completeMeditation() {
        isRunning = false
        timer?.cancel()

        // Guardar sesión completa
        saveMeditationProgress(totalDurationMinutes * 60)

        AlertDialog.Builder(this)
            .setTitle(";Meditación Completada! 🧘")
            .setMessage("Has completado $totalDurationMinutes minutos de meditación.\n\n¿Cómo te sientes?")
            .setPositiveButton("Bien") { _, _ ->
                recordMood("bien")
                finish()
            }
            .setNeutralButton("Muy bien") { _, _ ->
                recordMood("muy_bien")
                finish()
            }
            .setNegativeButton("Relajado") { _, _ ->
                recordMood("relajado")
                finish()
            }
            .setCancelable(false)
            .show()
    }

    private fun showStopConfirmation() {
        AlertDialog.Builder(this)
            .setTitle("Detener meditación")
            .setMessage(";Deseas detener la meditación ahora?")
    }
}

```

```
Pixel 9  app  MeditationSessionActivity.kt
class MeditationSessionActivity : AppCompatActivity() {
143
144     private fun completeMeditation() {
145         isRunning = false
146         timer?.cancel()
147
148         // Guardar sesión completa
149         saveMeditationProgress(totalDurationMinutes * 60)
150
151         AlertDialog.Builder(this)
152             .setTitle("¡Meditación Completada! 🧘")
153             .setMessage("Has completado $totalDurationMinutes minutos de meditación.\n\n¿Cómo te sientes?")
154             .setPositiveButton("Bien") { _, _ ->
155                 recordMood("bien")
156                 finish()
157             }
158             .setNeutralButton("Muy bien") { _, _ ->
159                 recordMood("muy_bien")
160                 finish()
161             }
162             .setNegativeButton("Relajado") { _, _ ->
163                 recordMood("relajado")
164                 finish()
165             }
166             .setCancelable(false)
167             .show()
168     }
169
170     private fun showStopConfirmation() {
171         AlertDialog.Builder(this)
172             .setTitle("Detener meditación")
173             .setMessage("¿Deseas detener la meditación ahora?")
174     }
175 }
```

Experiencia de Sesión Inmersiva

Una vez iniciada, la sesión activa un entorno diseñado para la relajación. Un sistema de fases actualiza las instrucciones cada minuto (ej. "Relaja los hombros") con transiciones de fade-in, permitiendo una progresión natural de la práctica.

Incluye un temporizador de gran formato y una barra de progreso. Un círculo central utiliza un ValueAnimator con movimiento infinito y un interpolador suave para guiar la respiración de forma rítmica.

```

private fun startMeditation() {
    isRunning = true
    isPaused = false
    buttonPlay.text = "⏸ Pausar"

    timer?.cancel()
    timer = object : CountDownTimer(remainingTimeMillis, 1000) {
        override fun onTick(millisUntilFinished: Long) {
            remainingTimeMillis = millisUntilFinished
            updateTimerDisplay()

            val secondsElapsed = totalDurationMinutes * 60 - (millisUntilFinished / 1000).toInt()
            progressBar.progress = (millisUntilFinished / 1000).toInt()

            // Cambiar instrucciones cada minuto
            if (secondsElapsed % 60 == 0 && secondsElapsed > 0) {
                updateInstruction()
            }
        }

        override fun onFinish() {
            completeMeditation()
        }
    }.start()

    Snackbar.make(rootView, "Meditación iniciada. Relájate...", Snackbar.LENGTH_SHORT).show()
}

```

Control y Gestión de Estados

Permite alternar entre reproducción y pausa. Para evitar interrupciones no deseadas, el botón de detener requiere una confirmación mediante un AlertDialog.

Al finalizar, se captura el estado emocional del usuario ("Relajado", "Bien"), lo que añade una capa de autoconocimiento al registro de progreso.

```

class MeditationSessionActivity : AppCompatActivity() {
    private fun completeMeditation() {
        AlertDialog.Builder(this)
            .setTitle("Meditación Completada! 🙏")
            .setMessage("Has completado $totalDurationMinutes minutos de meditación. ¿Cómo te sientes?")
            .setPositiveButton("Bien") { _, _ ->
                recordMood("bien")
                finish()
            }
            .setNeutralButton("Muy bien") { _, _ ->
                recordMood("muy_bien")
                finish()
            }
            .setNegativeButton("Relajado") { _, _ ->
                recordMood("relajado")
                finish()
            }
            .setCancelable(false)
            .show()
    }

    private fun showStopConfirmation() {
        AlertDialog.Builder(this)
            .setTitle("Detener meditación")
            .setMessage("¿Deseas detener la meditación ahora?")
            .setPositiveButton("Sí, detener") { _, _ ->
                stopMeditation()
            }
            .setNegativeButton("Continuar", null)
            .show()
    }
}

```

Análisis y Gamificación

El sistema monitorea el compromiso del usuario a través de la persistencia en SharedPreferences.

Registra el tiempo total meditado, el número de sesiones y la última fecha de actividad. Además, que al alcanzar hitos (como la primera o décima sesión), el sistema dispara automáticamente notificaciones de logro, reforzando la retención y la motivación.

```
// Verificar logro
if (totalSessions + 1 == 1) {
    NotificationHelper(this).sendAchievementNotification(
        "Primera Meditación",
        "¡Has completado tu primera sesión de meditación!"
    )
} else if (totalSessions + 1 == 10) {
    NotificationHelper(this).sendAchievementNotification(
        "Meditador Dedicado",
        "¡Has completado 10 sesiones de meditación!"
    )
}
}

private fun recordMood(mood: String) {
    val sharedPreferences = getSharedPreferences("MindCarePrefs", Context.MODE_PRIVATE)
    sharedPreferences.edit().putString("last_mood_after_meditation", mood).apply()

    Snackbar.make(rootView, "Estado registrado. ¡Gracias por meditar! 🧘", Snackbar.LENGTH_SHORT).show()
}
```

Mini-Juegos

El sistema integra tres experiencias diseñadas para la reducción de estrés mediante distracción positiva, utilizando mecánicas de estimulación sensorial y resolución de conflictos cognitivos.

Juego de Burbujas

Implementa una mecánica de *infinite runner* vertical para fomentar la coordinación ojo-mano.

- **Generación Dinámica:** Mediante `postDelayed` recursivo, el sistema genera burbujas con tamaños, colores y posiciones aleatorias para evitar patrones predecibles.
- **Animación y Limpieza:** Utiliza `ObjectAnimator` para el desplazamiento vertical. Al completarse o al ser "explotada", la vista se elimina del contenedor, garantizando una gestión eficiente de la memoria (*garbage collection*).

- **Feedback Visual:** Al tocar, una animación triple (escala y transparencia) simula el estallido, reforzada por una respuesta inmediata en el puntaje.

```

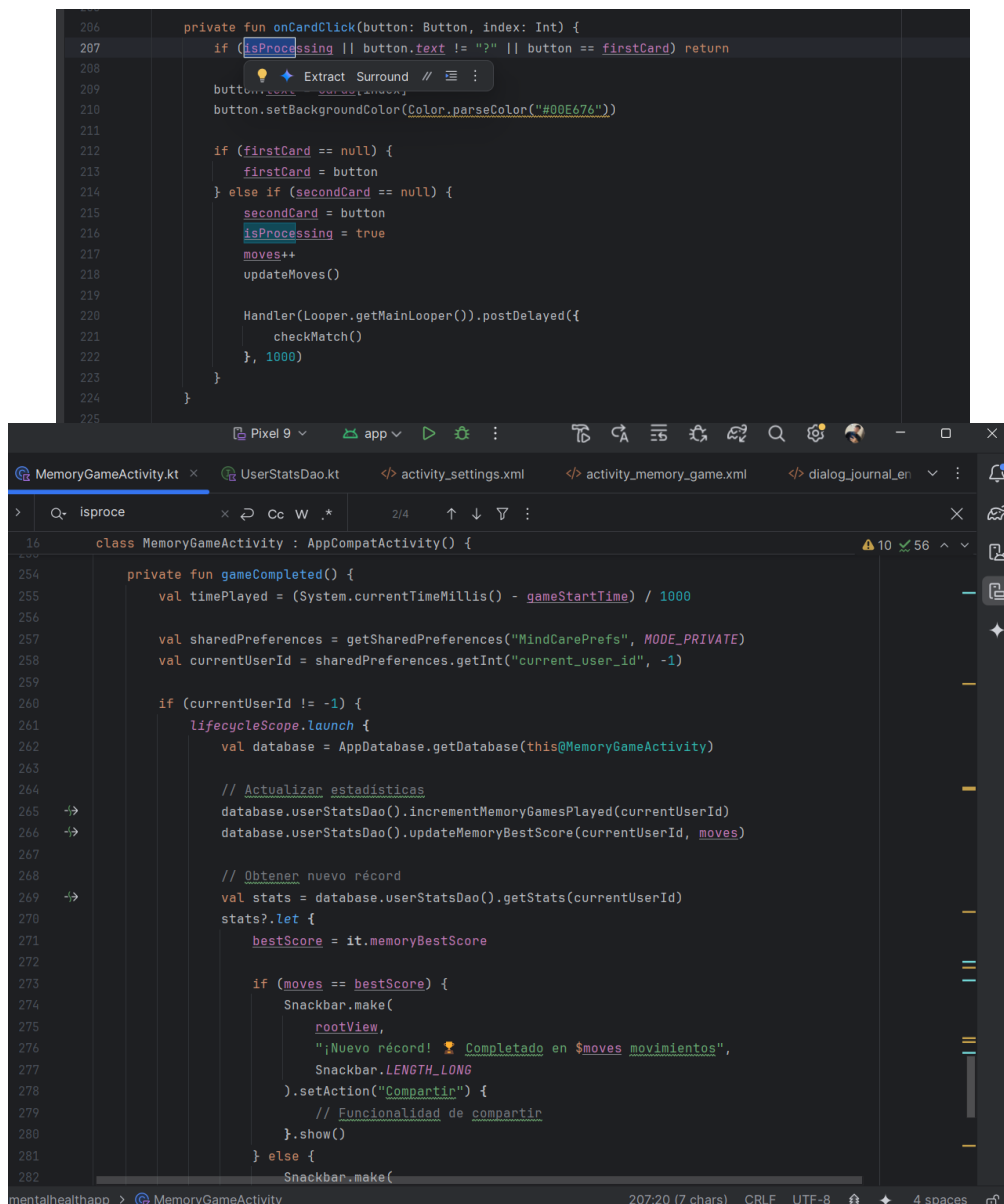
18 class BubbleGameActivity : AppCompatActivity() {
115     private fun createBubbles() {
142         popBubble(it)
143     }
144
145     gameContainer.addView(bubble)
146     animateBubble(bubble)
147
148     gameContainer.postDelayed({
149         createBubbles()
150     }, Random.nextLong(500, 1500))
151 }
152
153 private fun animateBubble(bubble: View) {
154     val animator = ObjectAnimator.ofFloat(
155         bubble,
156         "translationY",
157         bubble.y,
158         -300f
159     ).apply {
160         duration = Random.nextLong(3000, 6000)
161         addListener(object : AnimatorListenerAdapter() {
162             override fun onAnimationEnd(animation: Animator) {
163                 gameContainer.removeView(bubble)
164             }
165         })
166         start()
167     }
168 }
169
170 private fun popBubble(bubble: View) {
171     score += 10
172     bubblesPopped++
173     updateScore()
174
175     // Mostrar feedback cada 5 burbujas
176     if (bubblesPopped % 5 == 0) {
177         Snackbar.make(rootView, "${bubblesPopped} burbujas! 🎉", Snackbar.LENGTH_SHORT).show()
178     }
179
180     bubble.animate()
181         .scaleX(1.5f)
182         .scaleY(1.5f)
183         .alpha(0f)
184         .setDuration(200)
185         .withEndAction {
186             gameContainer.removeView(bubble)
187         }
188         .start()
189 }

```

Juego de Memoria

Un desafío clásico de tablero 4x4 centrado en la retención de información a corto plazo.

- **Lógica de Verificación:** Al seleccionar dos cartas, un Handler gestiona el tiempo de visualización antes de validar el par. Se implementa un bloqueo de entrada (isProcessing) para evitar que el usuario interactúe con una tercera carta durante la comparación.
- **Persistencia de Partida:** Es el juego con mayor robustez ante interrupciones; el sistema guarda la posición exacta de cada carta y cuáles han sido reveladas, permitiendo que una rotación de pantalla no reinicie el progreso.



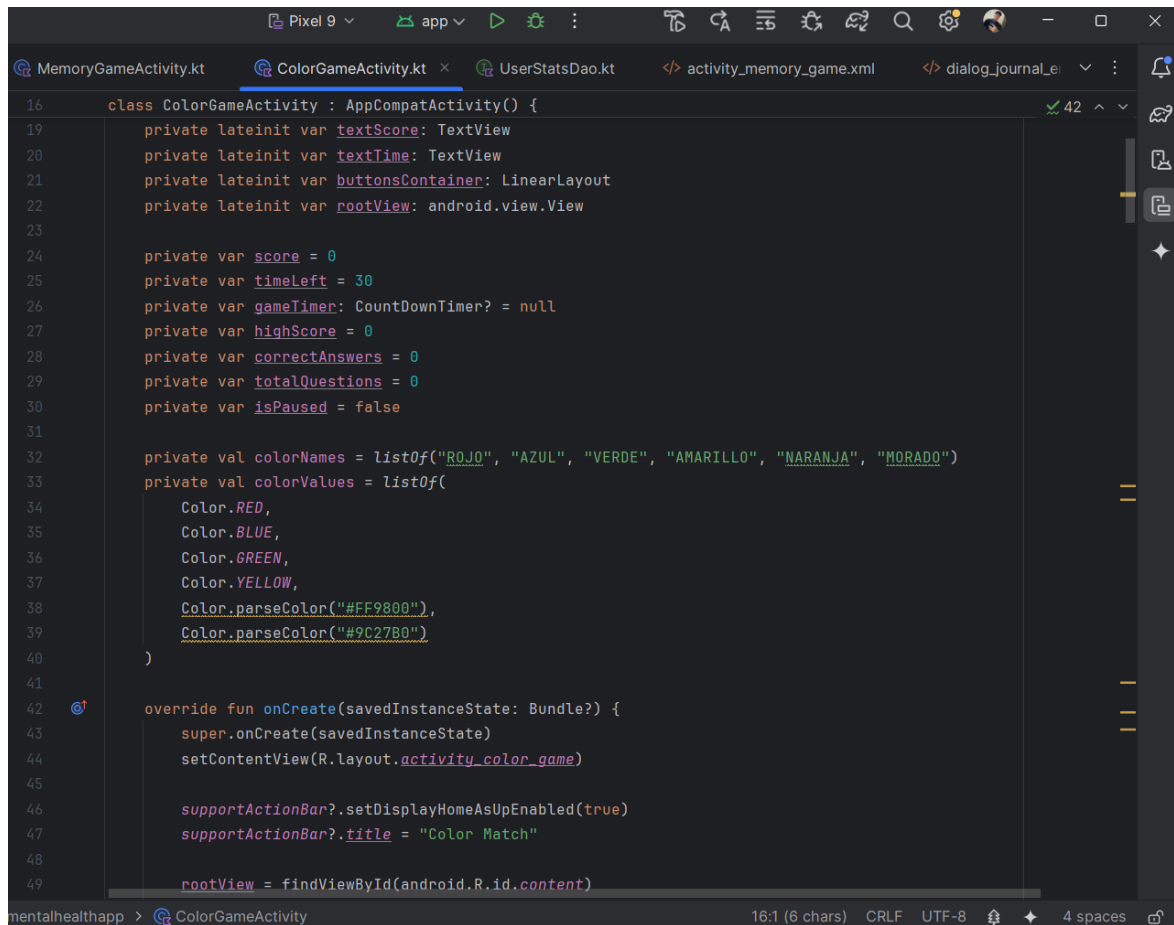
```
206 private fun onCardClick(button: Button, index: Int) {
207     if (isProcessing || button.text != "?" || button == firstCard) return
208
209     button.setOnClickListener {
210         button.setBackgroundColor(Color.parseColor("#00E676"))
211
212         if (firstCard == null) {
213             firstCard = button
214         } else if (secondCard == null) {
215             secondCard = button
216             isProcessing = true
217             moves++
218             updateMoves()
219
220             Handler(Looper.getMainLooper()).postDelayed({
221                 checkMatch()
222             }, 1000)
223         }
224     }
225 }

class MemoryGameActivity : AppCompatActivity() {
254 private fun gameCompleted() {
255     val timePlayed = (System.currentTimeMillis() - gameStartTime) / 1000
256
257     val sharedPreferences = getSharedPreferences("MindCarePrefs", MODE_PRIVATE)
258     val currentUserId = sharedPreferences.getInt("current_user_id", -1)
259
260     if (currentUserId != -1) {
261         lifecycleScope.launch {
262             val database = AppDatabase.getDatabase(this@MemoryGameActivity)
263
264             // Actualizar estadísticas
265             database.userStatsDao().incrementMemoryGamesPlayed(currentUserId)
266             database.userStatsDao().updateMemoryBestScore(currentUserId, moves)
267
268             // Obtener nuevo récord
269             val stats = database.userStatsDao().getStats(currentUserId)
270             stats?.let {
271                 bestScore = it.memoryBestScore
272
273                 if (moves == bestScore) {
274                     Snackbar.make(
275                         rootView,
276                         "¡Nuevo récord! 🏆 Completado en $moves movimientos",
277                         Snackbar.LENGTH_LONG
278                     ).setAction("Compartir") {
279                         // Funcionalidad de compartir
280                     }.show()
281                 } else {
282                     Snackbar.make(
```

Juego de Colores

Basado en el test neuropsicológico de Stroop, este juego entrena la flexibilidad cognitiva al separar la lectura de la percepción del color.

- **Interferencia Cognitiva:** El algoritmo asegura que el color del texto y la palabra escrita siempre difieran, forzando al usuario a ignorar el estímulo visual dominante.



```
16 class ColorGameActivity : AppCompatActivity() {
19     private lateinit var textScore: TextView
20     private lateinit var textTime: TextView
21     private lateinit var buttonsContainer: LinearLayout
22     private lateinit var rootView: android.view.View
23
24     private var score = 0
25     private var timeLeft = 30
26     private var gameTimer: CountdownTimer? = null
27     private var highScore = 0
28     private var correctAnswers = 0
29     private var totalQuestions = 0
30     private var isPaused = false
31
32     private val colorNames = listOf("ROJO", "AZUL", "VERDE", "AMARILLO", "NARANJA", "MORADO")
33     private val colorValues = listOf(
34         Color.RED,
35         Color.BLUE,
36         Color.GREEN,
37         Color.YELLOW,
38         Color.parseColor("#FF9800"),
39         Color.parseColor("#9C27B0")
40     )
41
42     override fun onCreate(savedInstanceState: Bundle?) {
43         super.onCreate(savedInstanceState)
44         setContentView(R.layout.activity_color_game)
45
46         supportActionBar?.setDisplayHomeAsUpEnabled(true)
47         supportActionBar?.title = "Color Match"
48
49         rootView = findViewById(android.R.id.content)
```

- **Sistema de Gamificación:**
 - **Puntuación:** Implementa bonos por racha y penalizaciones controladas (el puntaje nunca es negativo).
 - **Alertas de Tiempo:** El CountdownTimer dispara cambios visuales en la interfaz (colores de advertencia) cuando restan menos de 5 segundos, aumentando la intensidad de la ronda.

```

private fun startTimer() {
    gameTimer?.cancel()
    gameTimer = object : CountDownTimer((timeLeft * 1000).toLong(), 1000) {
        override fun onTick(millisUntilFinished: Long) {
            timeLeft = (millisUntilFinished / 1000).toInt()
            textTime.text = "🕒 Tiempo: ${timeLeft}s"

            // Advertencia cuando quedan 5 segundos
            if (timeLeft == 5) {
                Snackbar.make(rootView, "⚠️ ¡Solo 5 segundos!", Snackbar.LENGTH_SHORT)
                    .setBackgroundTint(Color.parseColor("#FF5722"))
                    .show()
            }
        }
    }

    override fun onFinish() {
        endGame()
    }
}.start()
}

```

```

private fun checkAnswer(selected: Int, correct: Int) {
    if (selected == correct) {
        score += 10
        correctAnswers++
        updateScore()

        // Bonus por racha
        if (correctAnswers % 5 == 0) {
            score += 25
            Snackbar.make(rootView, "¡RACHA! +25 puntos bonus 🟡", Snackbar.LENGTH_SHORT)
                .setBackgroundTint(Color.parseColor("#FF9800"))
                .show()
        }

        nextRound()
    } else {
        score -= 5
        if (score < 0) score = 0
        updateScore()

        Snackbar.make(rootView, "Incorrecto -5", Snackbar.LENGTH_SHORT)
            .setBackgroundTint(Color.parseColor("#F44336"))
            .show()
    }
}
}

```

Sistema de Estadísticas y Logros

El sistema de estadísticas de la aplicación funciona como un motor de retroalimentación en tiempo real que transforma cada interacción del usuario en datos significativos para su bienestar. Mediante una arquitectura de recolección inmediata, cualquier acción relevante incrementa contadores específicos en `SharedPreferences`, asegurando que el progreso se registre de forma persistente y asíncrona.

Por ejemplo, al guardar una entrada en el diario o completar una sesión de respiración, el sistema actualiza variables como el total de ciclos, las sesiones acumuladas y las rachas de uso sin interrumpir la experiencia de navegación.

La visualización de estos datos ocurre de manera dinámica en la pantalla principal. Cada vez que la actividad retoma el foco, el sistema evalúa si existen métricas acumuladas para presentar un resumen ejecutivo mediante un componente interactivo que invita al usuario a explorar un panel detallado. Este diálogo detallado organiza la información en categorías lógicas que abarcan desde el historial del diario y las sesiones de respiración hasta el desempeño técnico en los mini-juegos, incluyendo récords de puntuación y precisión. Además, se integra una funcionalidad de exportación social que permite compartir estos avances con otras aplicaciones del dispositivo mediante un formato de texto plano estructurado.

La gamificación se implementa a través de un sistema de logros basado en el patrón de diseño observador, donde las verificaciones se disparan por eventos específicos.

Este enfoque garantiza que las condiciones de desbloqueo se evalúen inmediatamente después de que el usuario completa una tarea, evitando procesos de fondo innecesarios. Para asegurar una experiencia de usuario gratificante, cada logro cuenta con banderas lógicas que previenen notificaciones duplicadas y utiliza canales de alta prioridad para celebrar el hito. Estas notificaciones emplean estilos de texto expandible para ofrecer mensajes motivacionales profundos que explican el impacto positivo de cada logro alcanzado en el camino hacia el bienestar mental.

```
private fun showDetailedStats(stats: com.example.mentalhealthapp.database.entities.UserStats) {
    val message = buildString {
        append("📊 ESTADÍSTICAS DETALLADAS\n\n")
        append("📅 DIARIO: ${stats.totalJournalEntries} entradas\n")
        append("🌬️ RESPIRACIÓN: ${stats.totalBreathingSessions} sesiones\n")
        append("🫧 BURBUJAS: ${stats.totalBubblesPopped} reventadas\n")
        append("🧠 MEMORIA: ${stats.memoryGamesPlayed} juegos\n")
        append("🎨 COLOR: ${stats.colorGamesPlayed} juegos")
    }

    AlertDialog.Builder(this)
        .setTitle("Tus Estadísticas")
        .setMessage(message)
        .setPositiveButton("Cerrar", null)
        .setNeutralButton("Compartir") { _, _ -> shareStats(message) }
        .show()
}
```

Manejo de Intents y Navegación

El sistema de comunicación entre componentes se fundamenta en el uso de Intents, los cuales permiten la navegación fluida y el intercambio de información entre las diversas actividades de la aplicación. Mediante el uso de Intents explícitos, el sistema define trayectorias de navegación interna claras, como el tránsito desde el menú principal hacia los módulos de bienestar o juegos. Este proceso se complementa con el paso de datos a través de extras, permitiendo que una actividad receptora, como la del diario, capture el estado de ánimo seleccionado previamente para pre-llenar títulos o almacenar metadatos contextuales, lo que optimiza la experiencia de usuario al evitar la redundancia de datos.

La arquitectura de navegación está diseñada para ser escalable y jerárquica, utilizando la propiedad de actividad padre en el manifiesto de Android para garantizar que el botón de retroceso superior funcione correctamente. Para gestionar la memoria y la seguridad del flujo, el sistema emplea métodos de finalización de actividades que limpian la pila de retroceso en momentos críticos, como después de un inicio de sesión exitoso o al cerrar la sesión, impidiendo que el usuario regrese a pantallas de autenticación mediante el botón físico del dispositivo.

Más allá de la comunicación interna, la aplicación implementa Intents implícitos para interactuar con el ecosistema global de Android. Esto se manifiesta en la funcionalidad de compartir, donde el sistema invoca un selector de aplicaciones para enviar estadísticas o entradas del diario a servicios externos como mensajería o correo electrónico. Asimismo, para resolver situaciones de permisos denegados, el sistema genera vínculos directos hacia la configuración de ajustes del dispositivo, facilitando al usuario la gestión de notificaciones sin necesidad de navegar manualmente por los menús del sistema operativo.

RESULTADOS

Registro del usuario

12:45 25%

MindCare



Salud Mental

Tu espacio de bienestar mental

Correo electrónico

Contraseña

INICIAR SESIÓN

[¿No tienes cuenta? Regístrate](#)

12:45 25%

MindCare

Crear cuenta

Crea tu cuenta

Comienza tu camino al bienestar

Nombre completo  Gabriela

Correo electrónico  gaby@gmail.com

Contraseña (min. 6 caracteres)  

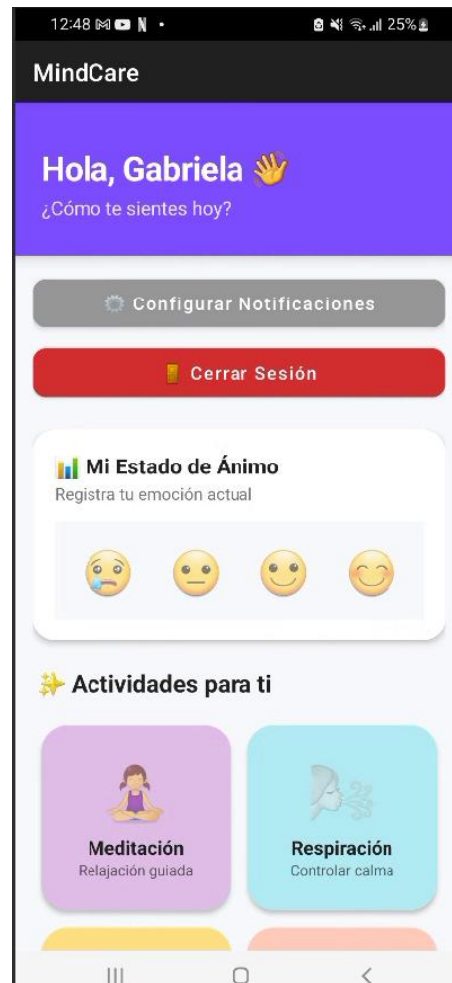
Confirmar contraseña  

CANCELAR REGISTRAR

[¿No tienes cuenta? Regístrate](#)

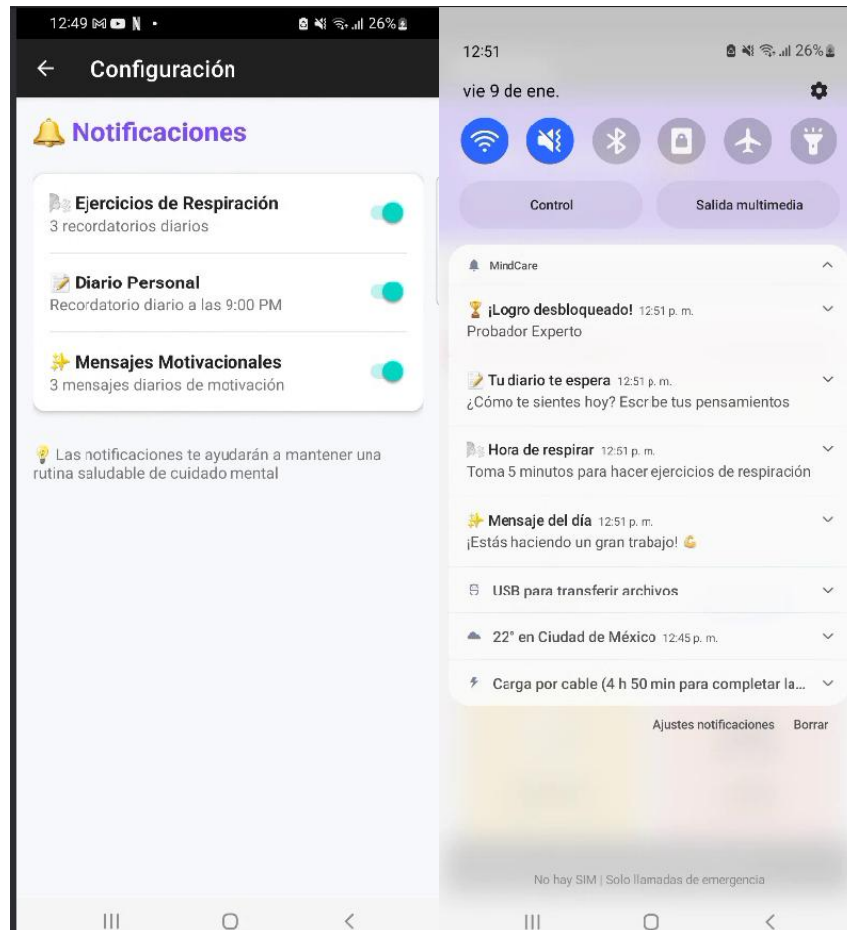
menú principal

Aquí el usuario puede elegir como se siente el día de hoy y se guarda en la base de datos



Configuración de notificaciones

El usuario activa o desactiva las notificaciones, recibirá recordatorios o logros en lo que vaya avanzando en la aplicación.



Sección de meditación

En este apartado se tienen meditaciones guiadas, donde se le da un contador para que se tome los minutos recomendados para completarlos.



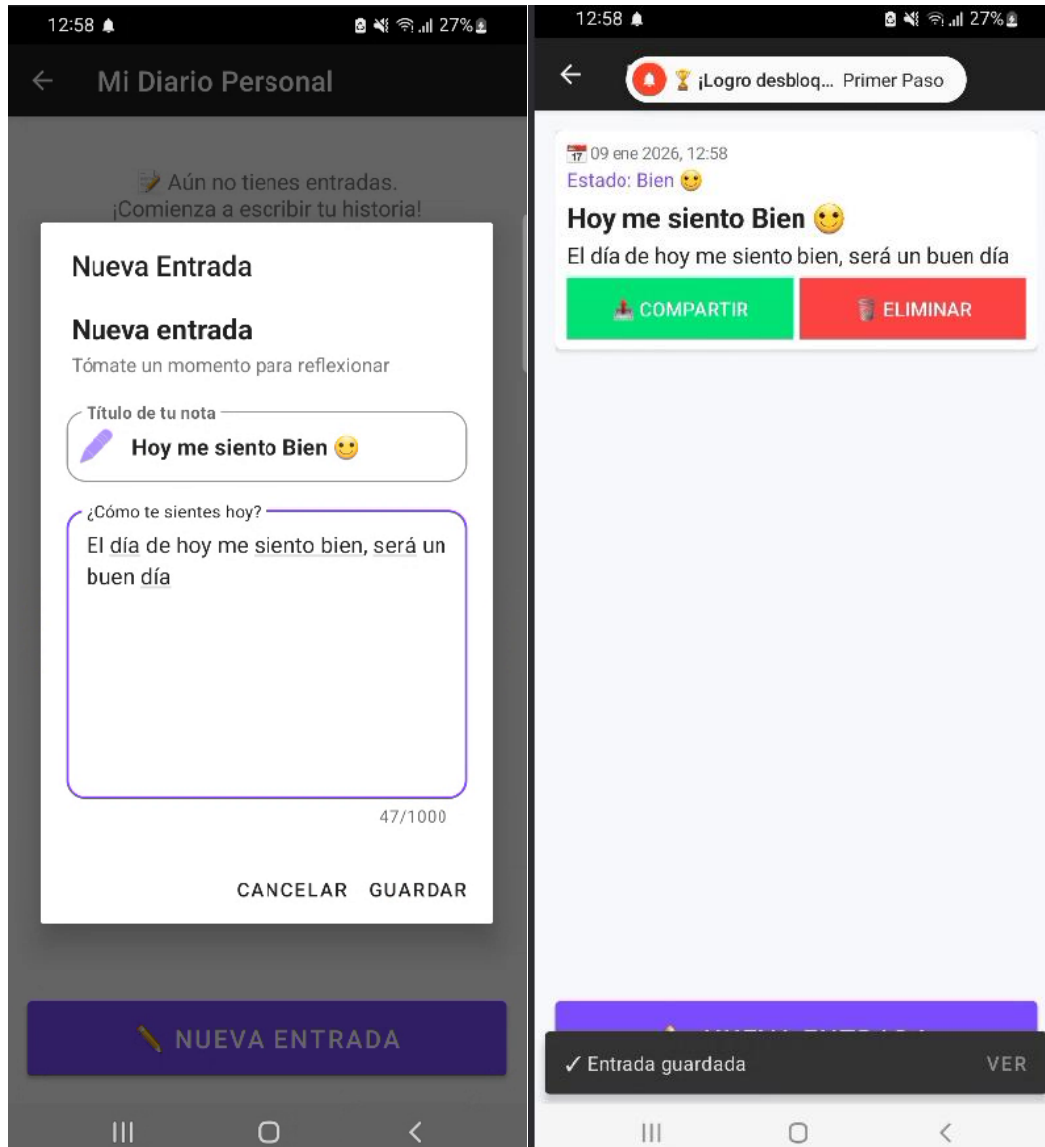
Sección de respiración

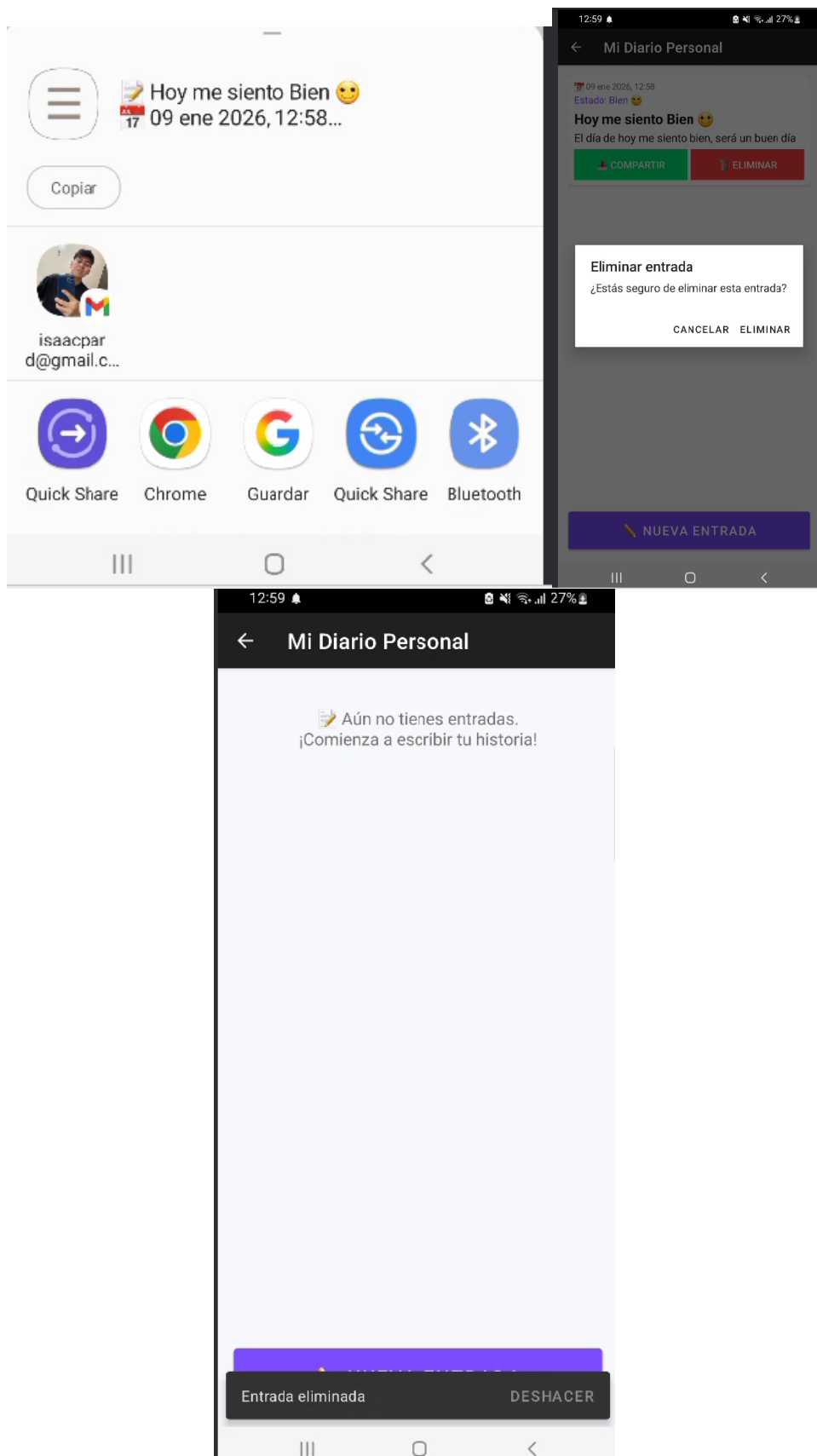
Aquí se aplica la técnica de 4-4-4. Donde inhala, sostiene y exhala para poder respirar de mejor manera y liberar el estrés que tenga el usuario.



Diario personal

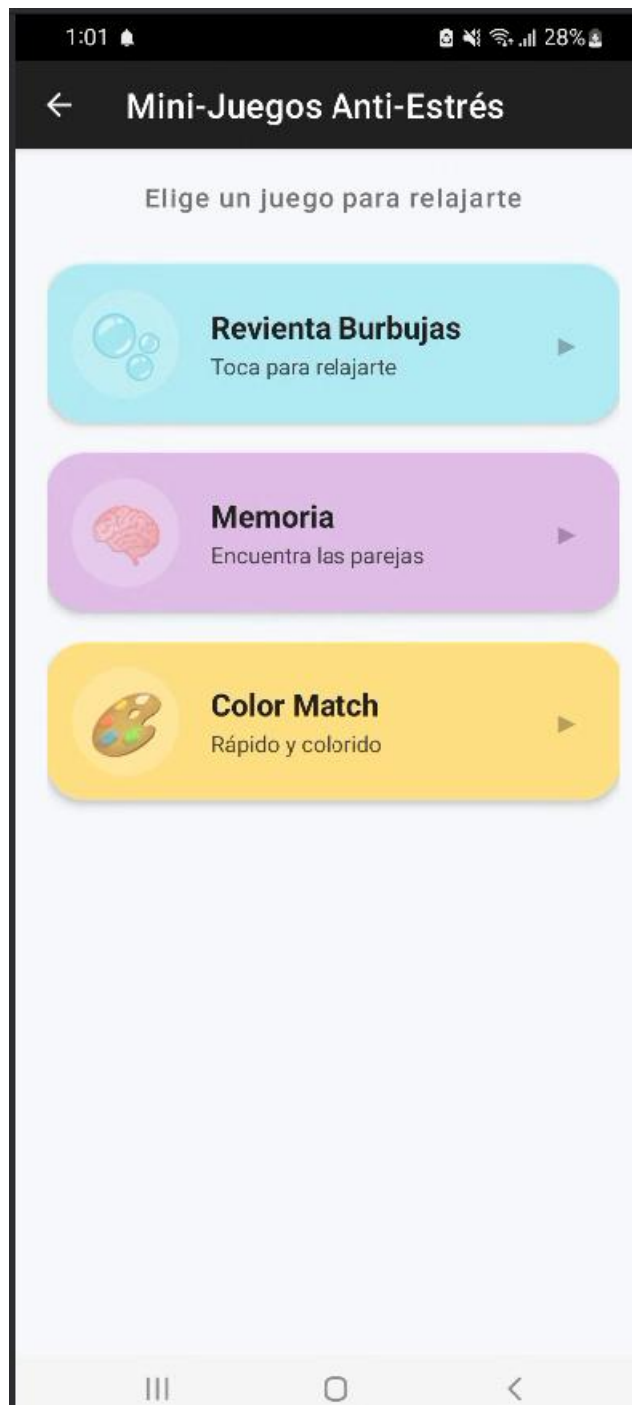
El usuario puede escribir notas, redactar como se siente el día de hoy y además de compartir dichas notas con alguna persona o si así lo desea también puede eliminarla.





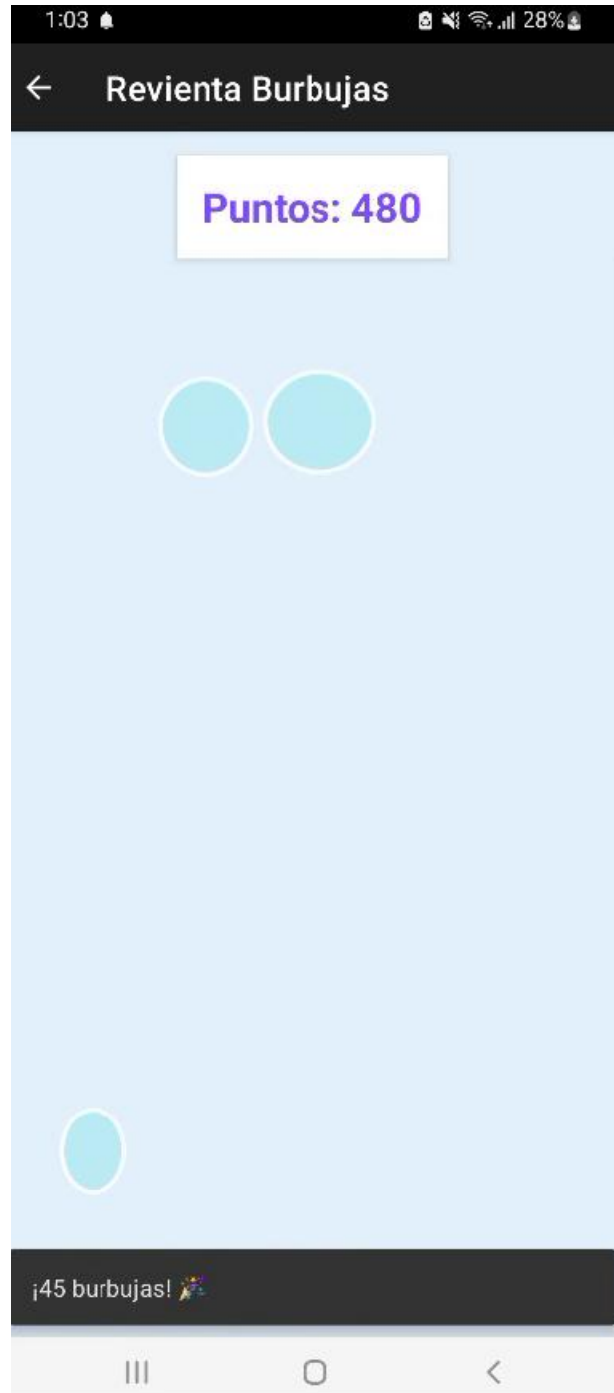
Sección de mini-juegos

Aquí el usuario puede elegir que jugar para poder desestresarse, el menú muestra 3.



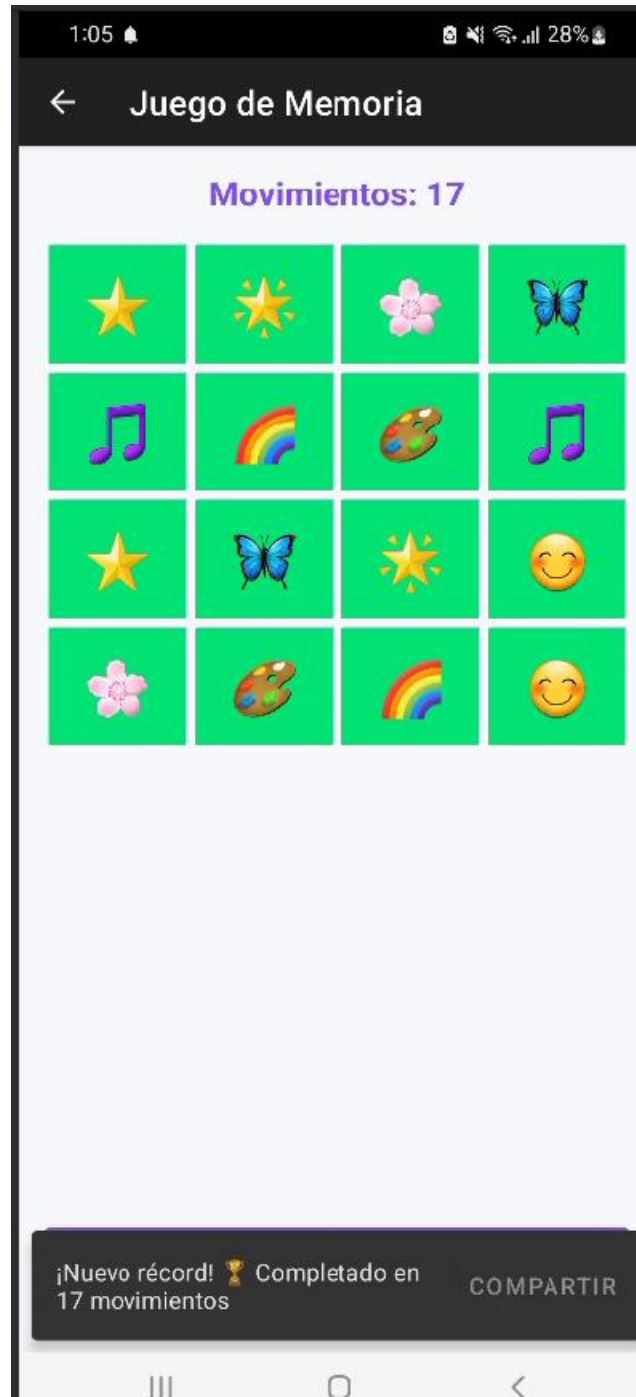
Juego de burbujas

Aquí el usuario debe explotar burbujas y lograr la mayor explosión de burbujas, este juego se hizo con la finalidad de que el usuario se distraiga y trate de liberar la tensión que tenga.



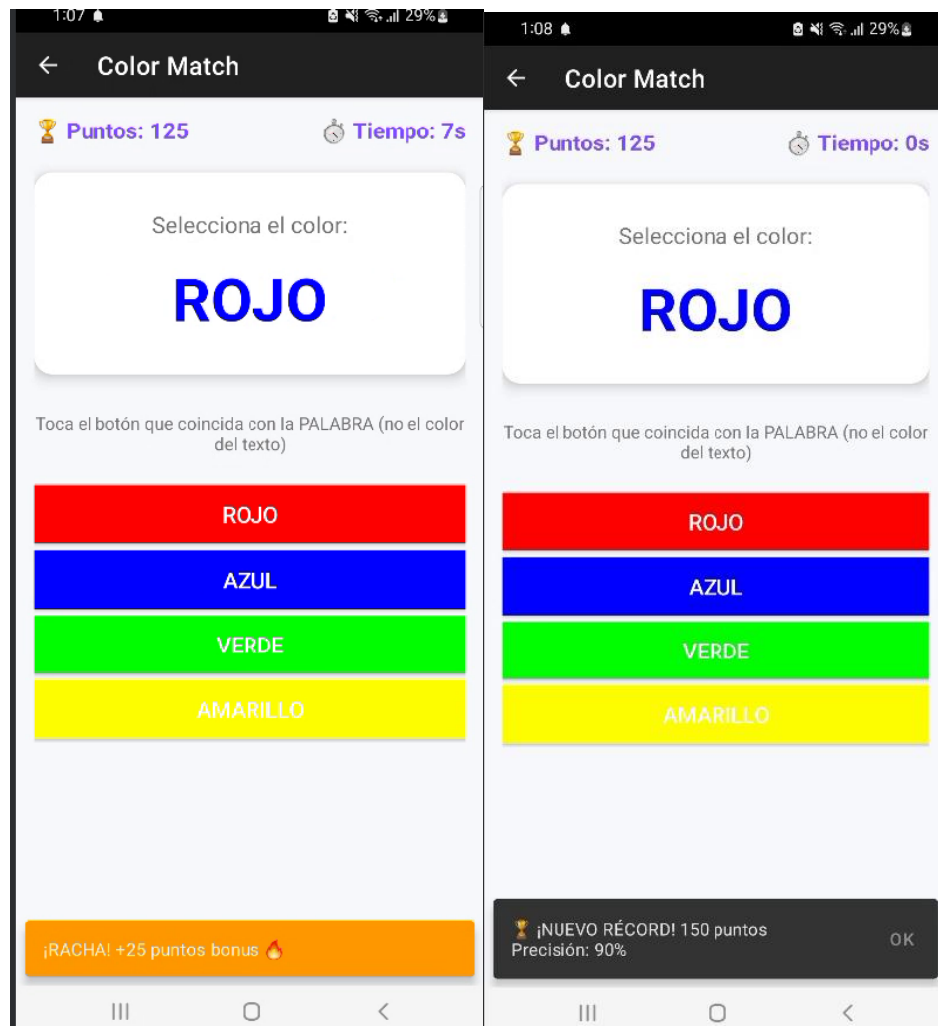
Juego de memoria

El usuario debe de encontrar la pareja y se registran sus movimientos, para que día con día vaya mejorando y lo haga con menos movimientos que la primera vez que lo hizo.



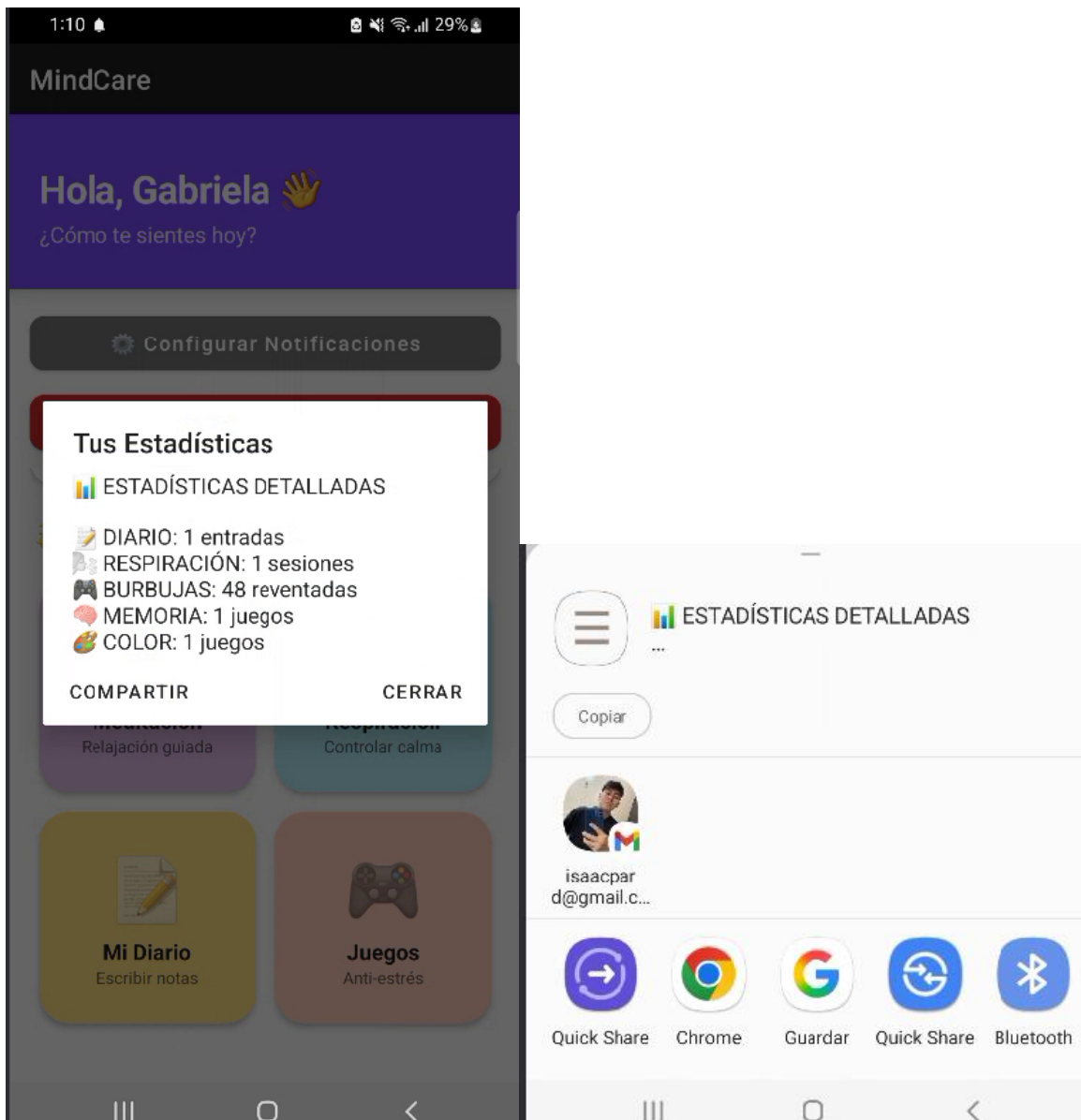
Juego de colores.

El usuario demostrará sus habilidades para poner el color correcto en donde debe de hacer lo mejor posible en 30 segundos, lo que no sabe, es que el nombre será diferente al color que corresponde, por lo que debe de ser astuto. Este juego ayuda a pensar y despertar su mente para su día a día.



Estadísticas

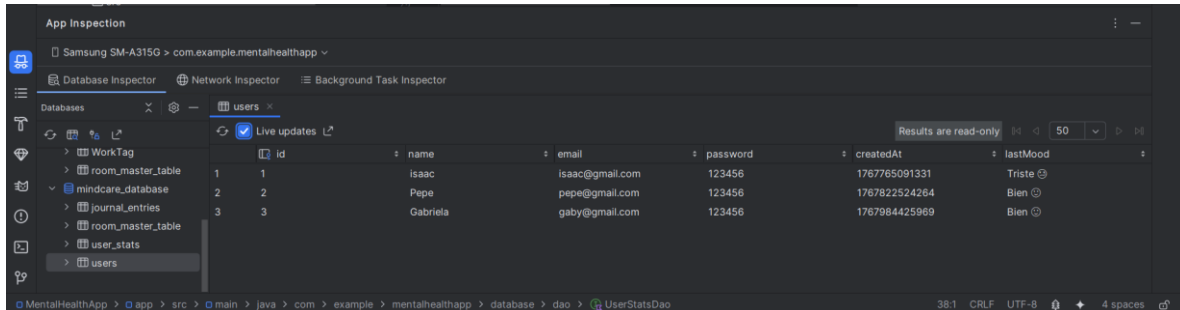
El usuario observa las estadísticas que tuvo en el día, además de observar lo que realizó y poder compartirlas igualmente.



Base de datos

Creación de usuarios

Se observa que el usuario se creó correctamente



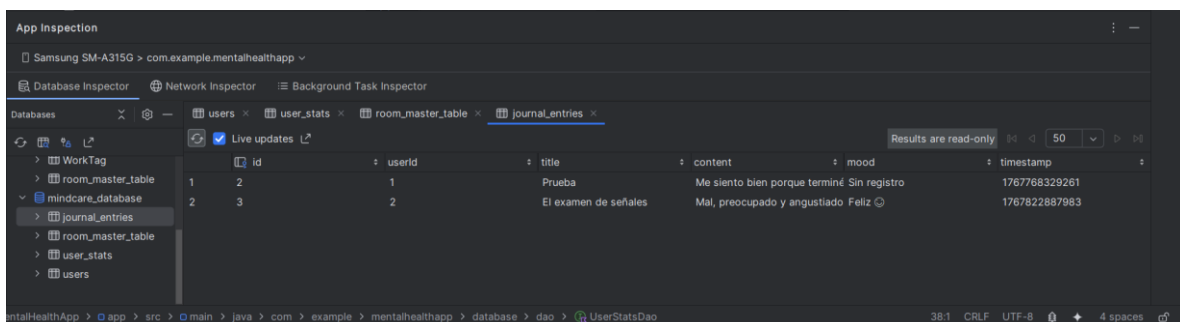
Estadísticas

Se guardar el progreso del usuario.



Diario

Registro de notas que el usuario haya realizado.



CONCLUSIÓN

“Salud mental” representa una implementación técnicamente sólida de una aplicación móvil de bienestar mental que combina principios de psicología positiva con desarrollo Android moderno. El proyecto demuestra dominio de conceptos fundamentales y avanzados del ecosistema Android incluyendo ciclo de vida de Activities, persistencia de datos, sistema de notificaciones programadas, animaciones, manejo de permisos en runtime, y arquitectura de software escalable.

Desde la perspectiva técnica, la aplicación exhibe calidad profesional mediante código limpio siguiendo convenciones de Kotlin, separación apropiada de responsabilidades, manejo robusto de errores, y consideración cuidadosa de edge cases como cambios de configuración y estados del ciclo de vida. La elección de tecnologías (Kotlin, Material Design, SharedPreferences/Room, Coroutines) refleja comprensión de las mejores prácticas actuales en desarrollo Android.

La arquitectura modular permite extensibilidad futura sin refactorización mayor. Agregar nuevos tipos de meditación requiere solo llamar addMeditationCard con nuevos parámetros. Implementar nuevos logros implica agregar condiciones al sistema existente. Nuevas notificaciones se integran mediante adición de métodos en NotificationHelper y scheduling en NotificationScheduler. Esta arquitectura facilita mantenimiento y evolución continua.

Desde la perspectiva de producto, “Salud mental” proporciona valor real al usuario mediante herramientas respaldadas por investigación psicológica: ejercicios de respiración controlada para activación parasimpática, escritura expresiva para procesamiento emocional, meditación mindfulness para reducción de ansiedad, y distracción positiva mediante juegos cognitivos. El sistema de notificaciones promueve adherencia convirtiendo uso esporádico en hábito regular.

El enfoque en privacidad mediante almacenamiento local completo sin requerimientos de servidor o transmisión de datos es particularmente importante dado el contexto de salud mental donde la privacidad es fundamental. Los usuarios pueden confiar que sus pensamientos más íntimos en el diario permanecen completamente privados en su dispositivo.